

The University of Manchester  
Department of Computer Science  
Project Report 2021

**Classification of Medical Images into  
AD, MCI and Healthy Brain**

Author: Anna Lackey

Supervisor: Dr. Fumie Costen

## **Abstract**

### **Classification of Medical Images into AD, MCI and Healthy Brain**

**Author: Anna Lackey**

This project devised a method for the classification of the three stages in the development of Alzheimer's disease (AD) from three dimensional scans of the brain. AD is irreversible and progressive, gradually destroying healthy neurons within the brain. Before an AD diagnosis, changes are already evident in the brain within a stage called mild cognitive impairment (MCI). This is the stage between healthy brain and AD and is treatable. If MCI can be detected, treatment can be prescribed to prevent the progression to AD. Hence, it is important to be able to accurately classify MCI against AD and healthy brain. This project employed an ensemble method of convolutional neural networks, each trained on an individual anatomical plane of the MRI scanned brain. The results were compounded to provide a classification that utilises the three dimensional views that come from the acquired MRI scans but avoids the complexities of processing a three dimensional image. The resulting model achieved an 80.7% (3s.f.) accuracy, with a 72.4% (3s.f.) accuracy within the MCI class.

**Supervisor: Dr. Fumie Costen**

### **Acknowledgements**

A huge thank-you to all involved, with a special thanks to Fumie whose support has been so encouraging. This project has been a delight!

# Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                              | <b>8</b>  |
| 1.1      | Motivation . . . . .                             | 8         |
| 1.2      | Aims and Objectives . . . . .                    | 8         |
| 1.2.1    | Aims . . . . .                                   | 8         |
| 1.2.2    | Objectives . . . . .                             | 9         |
| 1.3      | Report Structure . . . . .                       | 10        |
| <b>2</b> | <b>Background</b>                                | <b>12</b> |
| 2.1      | Alzheimer's Disease (AD) . . . . .               | 12        |
| 2.1.1    | Causes . . . . .                                 | 12        |
| 2.1.2    | Stages - From NC to AD (MCI) . . . . .           | 13        |
| 2.1.3    | Diagnosis . . . . .                              | 14        |
| 2.1.4    | Treatment . . . . .                              | 15        |
| 2.2      | Machine Learning (ML) . . . . .                  | 16        |
| 2.2.1    | Machine Learning Pipeline . . . . .              | 16        |
| 2.2.2    | Classification and Supervised Learning . . . . . | 17        |
| 2.2.3    | Data . . . . .                                   | 17        |
| 2.2.4    | Model . . . . .                                  | 18        |
| 2.2.5    | Hyperparameters . . . . .                        | 21        |
| 2.2.6    | Evaluation . . . . .                             | 23        |
| <b>3</b> | <b>Previous Work</b>                             | <b>26</b> |
| 3.1      | Literary Review . . . . .                        | 26        |
| 3.1.1    | Preprocessing - Model Input . . . . .            | 26        |
| 3.1.2    | Classification Methods . . . . .                 | 29        |
| 3.2      | Implementation of Previous Methods . . . . .     | 33        |
| 3.2.1    | Data - Ethics . . . . .                          | 33        |
| 3.2.2    | Data - Preprocessing . . . . .                   | 34        |
| 3.2.3    | Data - 2.5D Slice Extraction . . . . .           | 35        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>4</b> | <b>Research Methods</b>                                      | <b>38</b> |
| 4.1      | Methods . . . . .                                            | 38        |
| 4.2      | Initial Model . . . . .                                      | 38        |
| 4.3      | Considerations . . . . .                                     | 39        |
| <b>5</b> | <b>Preprocessing</b>                                         | <b>40</b> |
| 5.1      | Image Size and Cropping . . . . .                            | 40        |
| 5.2      | Slice Point . . . . .                                        | 40        |
| 5.3      | Plane Size . . . . .                                         | 43        |
| 5.4      | Class Imbalance . . . . .                                    | 45        |
| 5.4.1    | Minimising . . . . .                                         | 45        |
| 5.4.2    | Maximising . . . . .                                         | 45        |
| 5.5      | Data Augmentation . . . . .                                  | 48        |
| 5.6      | Data Augmentation - Brightness Only on Final Model . . . . . | 49        |
| 5.7      | Preprocessing Results . . . . .                              | 49        |
| <b>6</b> | <b>Model Construction</b>                                    | <b>51</b> |
| 6.1      | Model Hyperparameters . . . . .                              | 51        |
| 6.1.1    | Ensemble . . . . .                                           | 51        |
| 6.1.2    | Random Search Model Optimisation . . . . .                   | 53        |
| 6.1.3    | Batch Normalisation . . . . .                                | 56        |
| 6.1.4    | Dropout . . . . .                                            | 58        |
| 6.1.5    | Network Depth . . . . .                                      | 59        |
| 6.1.6    | Filters . . . . .                                            | 61        |
| 6.1.7    | Kernel Size . . . . .                                        | 62        |
| 6.1.8    | Padding . . . . .                                            | 62        |
| 6.1.9    | Pooling . . . . .                                            | 63        |
| 6.1.10   | Activation Functions . . . . .                               | 64        |
| 6.2      | Training Hyperparameters . . . . .                           | 66        |
| 6.2.1    | Early Stopping . . . . .                                     | 66        |
| 6.2.2    | Epochs . . . . .                                             | 67        |
| 6.2.3    | Learning Rate and Batch Size . . . . .                       | 67        |
| 6.3      | Model Construction Results . . . . .                         | 69        |
| <b>7</b> | <b>Results and Discussion</b>                                | <b>70</b> |
| 7.1      | Results . . . . .                                            | 70        |
| 7.1.1    | Training . . . . .                                           | 70        |
| 7.1.2    | Metrics . . . . .                                            | 71        |
| 7.1.3    | Confusion Matrix . . . . .                                   | 72        |
| 7.2      | Discussion . . . . .                                         | 73        |

|                                                                 |            |
|-----------------------------------------------------------------|------------|
|                                                                 | 4          |
| 7.2.1 Discussion and Appraisal of Results and Methods . . . . . | 73         |
| 7.2.2 Evaluation of Objectives . . . . .                        | 74         |
| 7.2.3 Reflection . . . . .                                      | 75         |
| <b>8 Conclusion and Future Work</b>                             | <b>76</b>  |
| 8.1 Conclusions . . . . .                                       | 76         |
| 8.2 Future Work . . . . .                                       | 77         |
| <b>A 2.5D Data Extraction</b>                                   | <b>84</b>  |
| <b>B Adam Algorithm</b>                                         | <b>93</b>  |
| <b>C Initial Model</b>                                          | <b>94</b>  |
| <b>D Initial Tuner Model</b>                                    | <b>95</b>  |
| <b>E Ensemble Methods</b>                                       | <b>99</b>  |
| <b>F Models Resulting From Random Search</b>                    | <b>101</b> |
| <b>G Model Resulting from Conv Depth Experiment</b>             | <b>104</b> |
| <b>H Model Resulting from Dense Layer Depth Experiment</b>      | <b>105</b> |
| <b>I Final Model</b>                                            | <b>107</b> |

# List of Figures

|     |                                                                                                                                                                         |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | The creation of $A\beta$ protein by secretease enzymes. These fragments cause synaptic damage and are the building block of amyloid plaques [Kumar and Clark, 2005] . . | 13 |
| 2.2 | A brain demonstrating AD, showing cortical atrophy- white matter loss and dilated ventricles. [Cross and Underwood, 2013] . . . . .                                     | 15 |
| 2.3 | The Structure of an Artificial Neuron [Oduncuoglu and Kurt, 2015] . . . . .                                                                                             | 18 |
| 2.4 | A Single Hidden Layer Neural Network and a Deep Neural Network with 4 hidden layers . . . . .                                                                           | 20 |
| 2.5 | Operation of a Kernel on an Input to Produce an Output Feature Map [Ganesh, 2019]                                                                                       | 21 |
| 2.6 | Example Activation Functions [Sze et al., 2017] . . . . .                                                                                                               | 23 |
| 2.7 | Under and Over Fitting on a Binary Classification Problem [Patel, 2018] . . . . .                                                                                       | 23 |
| 3.1 | 246 Regions used as input for Gupta et al. [2019] . . . . .                                                                                                             | 27 |
| 3.2 | Anatomical Planes. Purves et al. [2012] . . . . .                                                                                                                       | 28 |
| 3.3 | Segmentation and Region of Interest Extraction of the three dimensional MRI Scans - Shown as three two dimensional images . . . . .                                     | 34 |
| 3.4 | The Visual difference when including multiple slices along an axis as opposed to singular slices . . . . .                                                              | 36 |
| 3.5 | Resulting Image of a single scan after saving as 1D array, reading it in and reshaping                                                                                  | 36 |
| 4.1 | Example Model Input Image . . . . .                                                                                                                                     | 38 |
| 4.2 | Random Search for configuring network hyperparameters Bergstra and Bengio [2012] . . . . .                                                                              | 38 |
| 4.3 | Neural Network Structure . . . . .                                                                                                                                      | 39 |
| 5.1 | Image Resize Experiment . . . . .                                                                                                                                       | 41 |
| 5.2 | Slice Point Experiment . . . . .                                                                                                                                        | 42 |
| 5.3 | Incorporating Multiple Slices from a Single slice to 10 to 30 to 60 ( Left to Right) .                                                                                  | 43 |
| 5.4 | Plane Size Experiment . . . . .                                                                                                                                         | 44 |
| 5.5 | Class Imbalance Experiment - Minimising a class . . . . .                                                                                                               | 46 |
| 5.6 | Class Imbalance Experiment - Maximising a class . . . . .                                                                                                               | 47 |
| 5.7 | Varying the Amount of Training Data Augmentation . . . . .                                                                                                              | 48 |
| 5.8 | Training Data Augmentation - Brightness . . . . .                                                                                                                       | 49 |

|      |                                                                    |    |
|------|--------------------------------------------------------------------|----|
| 6.1  | Ensemble Model Comparison . . . . .                                | 52 |
| 6.2  | Test Accuracies of All Network Topologies . . . . .                | 54 |
| 6.3  | Training and Validation Accuracy of Models over Training . . . . . | 55 |
| 6.4  | Training and Validation Loss of Models over Training . . . . .     | 55 |
| 6.5  | Training of Model with No Batch Normalisation . . . . .            | 56 |
| 6.6  | Training of Model with Single Batch Normalisation Layer . . . . .  | 56 |
| 6.7  | Test Accuracy when Varying Batch Norm Placement . . . . .          | 57 |
| 6.8  | Changing Depth in the Conv-Pool layers Results . . . . .           | 59 |
| 6.9  | Changing Depth in the Dense layers Results . . . . .               | 60 |
| 6.10 | Varying the Number of Filters . . . . .                            | 61 |
| 6.11 | Changing Kernel Size Results . . . . .                             | 62 |
| 6.12 | Changing the Activation Functions . . . . .                        | 65 |
| 6.13 | Early Stopping - Varying Patience . . . . .                        | 66 |
| 6.14 | Varying the Number of Epochs . . . . .                             | 67 |
| 6.15 | Varying the Learning Rate . . . . .                                | 67 |
| 6.16 | Changing Batch Size Results . . . . .                              | 68 |
| 7.1  | Final Model Training, Randomly Selected Fold . . . . .             | 71 |
| 7.2  | Confusion Matrix . . . . .                                         | 73 |

# List of Tables

|     |                                                                                   |    |
|-----|-----------------------------------------------------------------------------------|----|
| 2.1 | Performance Metrics for a Machine Learning Model. . . . .                         | 24 |
| 3.1 | A Summary of past work in the classification of AD . . . . .                      | 32 |
| 6.1 | A Summary of the performance of each model trained separately by plane . . . . .  | 51 |
| 6.2 | A Summary of the performance of each model . . . . .                              | 52 |
| 6.3 | The Hyperparameters Changed in Random Search . . . . .                            | 53 |
| 6.4 | Accuracy Performance for Each Plane from each model . . . . .                     | 54 |
| 6.5 | Accuracy Performance when Varying Dropout Amounts . . . . .                       | 58 |
| 6.6 | A Summary of the performance of each model with varying dense layer depth (3s.f.) | 60 |
| 6.7 | Models and their Number of Parameters for Filter Amount Experiment . . . . .      | 61 |
| 6.8 | A Summary of the performance of each model with varying pooling sizes (3s.f.) . . | 63 |
| 7.1 | Final Model Evaluation Metrics . . . . .                                          | 72 |

# **Chapter 1**

## **Introduction**

### **1.1 Motivation**

The severity and terminality of Alzheimer's Disease (AD) founds the motivations of the project. Detecting the Mild Cognitive Impairment (MCI) stage of its' progression gives a possibility of treatment that would prevent the deterioration to AD. This would drastically improve the quality of life and the prognosis for patients as well as relieving distress for the family. Unfortunately, distinguishing this stage with the human eye is incredibly difficult and MCI itself is not a diagnosis that a clinician would make; the diagnosis a patient would receive would be AD once the disease has progressed to that point. By employing Machine Learning (ML) methods, this stage can not only be recognised, but done so to a good accuracy.

### **1.2 Aims and Objectives**

#### **1.2.1 Aims**

This project aimed to devise a method for preprocessing MRI scans of the brain to accurately capture the data required for the classification of the data into three categories: AD, MCI and NC and to use ML methods to reliably perform this classification. A focus was maintained on the MCI stage, as this is the hardest classification to make and the one that can provide the most benefit to patients.

### 1.2.2 Objectives

The objectives encompass the three main stages of a ML project as well as the two main challenges faced within this application.

#### 1. Data Preprocessing

- (a) Apply and improve upon proven techniques to transform the three dimensional MRI scans into two dimensional data that can be used as input to a CNN model.
- (b) Criteria for success: The input to the model should be two dimensional, and the model should be able to perform the classification effectively using these.

#### 2. CNN

- (a) Devise and create a successful CNN architecture that can accurately perform the classification of the brain scans into AD, MCI and NC. This should train on a training data set and learn the patterns to perform the classification such that it can then work as effectively on unseen data.
- (b) Criteria for success: The accuracy of the model should be above that of previous years, that was 76.3% in Guo [2020] . The training and test accuracy should be within 5% of each other.

#### 3. MCI Performance

- (a) The detection of the MCI class should be prioritised as this is the point at which a diagnosis could lead to treatment to prevent the onset of AD. Within the project the focus should be on this classification.
- (b) Criteria for success: Calculate the class accuracies for the performance of the model in each class. MCI should be the highest.

#### 4. Small Dataset

- (a) A common challenge when applying ML to medical challenges is a small dataset. Being able to create a model that can learn to perform the classification despite the dataset being small is an important aspect of the project.
- (b) Criteria for success: Use just the dataset given

The end result of the project should consist of a method of preprocessing MRI scanned brains to input to a ML model, which is capable of performing an accurate classification of the brain into an AD, MCI or NC category.

## 1.3 Report Structure

This report has been structured with the intent of minimising upon any repetition and to present each concept thoroughly before it is referenced within the work.

### Chapter 2: Background

The background is subdivided into two sections:

- Alzheimer's Disease (AD)

Here a study of AD, its stages and its causes is given. In the presentation of its development, the MCI stage is emphasised, as well as the treatment and benefits of identifying this stage.

- Machine Learning (ML)

ML is defined and the methods and processes it entails are given. This establishes the structure in which the project work was undertaken: data preprocessing, model creation and model evaluation. It also defines a number of the methods and techniques involved, as well as the hyperparameters to be optimised. The presented concepts include CNNs, loss and activation functions, layer types and validation and evaluation methods.

### Chapter 3: Previous Work

This section presents the previous work undertaken into the uses of ML methods to identify and classify AD and its stages. First is a review of methods for the preprocessing and model creation currently divulged in the literature. This is followed by a further analysis of the previous work of Dai [2019], whose preprocessing methods provided a basis for the development of the preprocessing methods outlined for this project. The work of Dai [2019] is analysed and evaluated, giving a clear indication of its influence and how it was improved and built upon. As part of this, the dataset is introduced.

### Chapter 4: Research Methods

The methods and environment under which the project was conducted are given, including the method by which a baseline CNN structure was established. This model was used for conducting the experiments of the project.

## **Chapter 5: Preprocessing**

These experiments established the methods of data preprocessing applied to transform the scans into input for the final model. This included the analysis of where to extract from the three dimensional scans, as well as methods for dealing with class imbalance.

## **Chapter 6: Model Construction**

A number of investigations were undertaken to observe the effects of changing aspects of the CNN, referred to as hyperparameters. These hyperparameters include the depth of the model, different activation functions as well as parameters such as learning rate. This determined the final model architecture and the values of its user set hyperparameters.

## **Chapter 7: Results and Discussion**

The results entails a thorough evaluation of the final model, presenting an analysis of the training and using a number of evaluation metrics to examine its performance and behaviour.

The discussion outlines the meanings carried in the results and an evaluation of the projects methods and achievements.

## **Chapter 8: Conclusion and Future Work**

The conclusion presents the main findings and discoveries of the project.

Future work describes both the investigations not conducted as part of this project and the additional ones that arise from it.

# Chapter 2

## Background

### 2.1 Alzheimer's Disease (AD)

AD is a fatal neurodegenerative disease [Hames and Hooper, 2011]. It is a cause of dementia- a group of medical conditions with progressive deterioration in brain function [Young et al., 2014]. AD accounts for 70% of all cases of dementia in adults over 65 [Cross and Underwood, 2013] and 34% of all cases of dementia in adults under 65 [Kumar and Clark, 2005]. In the UK, AD itself affects 5% of people over the age of 65 and 15% of those over 80 [Cross and Underwood, 2013]. It has been coined the 'silent epidemic' as there is a concomitant increase in the number of cases as a result of an ageing population [Cross and Underwood, 2013].

#### 2.1.1 Causes

The definitive cause of AD is unknown, but the molecular pathology of the disease is clear [Kumar and Clark, 2005]. AD is caused by protein misfolding. The misfolding protein is *amyloid-beta* ( $A\beta$ ) [Hames and Hooper, 2011]. This is deposited in the cortex of the brain, leading to a build up of amyloid plaque [Kumar and Clark, 2005]. These protein aggregates damage the synapses, leading to the death of the surrounding neurons [Kumar and Clark, 2005, Hames and Hooper, 2011].

The amyloid precursor protein (APP) is processed by secretase enzymes to form the damaging  $A\beta$  proteins [Naish et al., 2009][Kumar and Clark, 2005]. Hence, mutations in the genetic code for APP are also a cause of AD [Kumar and Clark, 2005]. This process is shown in Figure 2.1: The creation of  $A\beta$ , taken from [Kumar and Clark, 2005].

Tangles similarly result in the death of the surrounding neurons. Tangles are bundles of filaments of a different protein - tau. They are located in the cytoplasm of neurons and the number of tangles has been shown to correlate with the severity of the dementia [Naish et al., 2009].

Tau tangles and amyloid deposits correlate as features of both AD and MCI [Dani et al., 2019].

Being a direct relative of someone with AD doubles a person's risk of developing the disease [Kumar and Clark, 2005], nonetheless most cases occur sporadically [Cross and Underwood, 2013].

### 2.1.2 Stages - From NC to AD (MCI)

The limbic system (entorhinal cortex and hippocampus) shows the first histological changes in AD, followed by the cerebral cortex [Cross and Underwood, 2013]. Histological changes are changes that can be observed when viewing tissue under a microscope. The severity of the changes correlates with the progression of the disease. [Cross and Underwood, 2013]

MCI is an intermediate state between NC and AD [Kumar and Clark, 2005]. The primary symptom is mild memory impairment, which exhibits itself to a greater extent than would be expected for the person's age, but not such that it would lead to a diagnosis of dementia [Kumar and Clark, 2005]. This mild memory impairment is often the only symptom of MCI [Kumar and Clark, 2005]. Hence MCI considered a 'state of uncertainty for the clinician', rather than a condition of a patient [Smith, 2013]. MCI does not progress to AD in all cases, but all cases of AD have gone through this stage [Gupta et al., 2019].

#### Criteria for MCI

A study by Jak et al. [2009] found that in the same sample of subjects, the rate of prevalence of MCI varied from 11% to 74% depending on the criteria for MCI used [Smith, 2013]. The data used for the project originates from the Alzheimer's Disease Neuroimaging Initiative (ADNI)<sup>1</sup>. ADNI

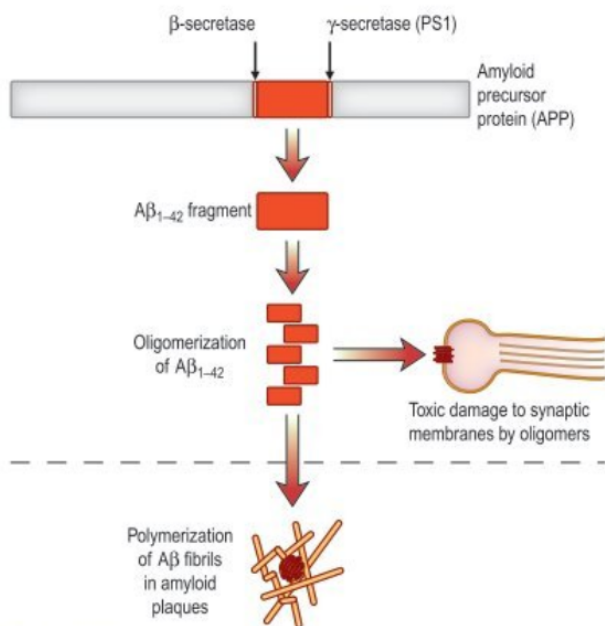


Figure 2.1: The creation of Aβ protein by secretase enzymes. These fragments cause synaptic damage and are the building block of amyloid plaques [Kumar and Clark, 2005]

<sup>1</sup>Data used in preparation of this article were obtained from the Alzheimer's Disease Neuroimaging Initiative (ADNI) database (adni.loni.usc.edu). As such, the investigators within the ADNI contributed to the design and implementation of ADNI and/or provided data but did not participate in analysis or writing of this report. A complete listing of ADNI investigators can be found at: [http://adni.loni.usc.edu/wp-content/uploads/how\\_to\\_apply/ADNI\\_Acknowledgement\\_List.pdf](http://adni.loni.usc.edu/wp-content/uploads/how_to_apply/ADNI_Acknowledgement_List.pdf)

uses the criteria defined by Petersen et al. [1997], but defines it operationally. The Petersen et al. [1997] criteria and ADNI operational definition for MCI are listed below:

- Complaint of memory disturbance  
ADNI: complaint of memory disturbance
- Objective evidence of a memory deficit  
ADNI: -1.5 standard deviations or more impairment on the delayed recall of Story A in the Wechslers memory scales logical memory subtest
- Preserved cognitive functions  
ADNI: Score of 24-30 on the Mini-Mental State Examination (MMSE)
- Intact activities of daily living  
ADNI: global Clinical Dementia Rating (CDR) score of 0.5
- Absence of dementia  
ADNI: consensus diagnosis of the absence of dementia

[Smith, 2013, Petersen et al., 1997]

### **2.1.3 Diagnosis**

#### **AD Clinical Diagnosis**

For a definitive diagnosis of AD as the cause of dementia, histopathological investigations are required (this is when the tissue is examined under a microscope) [Kumar and Clark, 2005]. However, in practice the clinical features are sufficient for a diagnosis [Kumar and Clark, 2005]. When observing memory impairment, the clinician looks for the effects that AD has on the episodic memory- this is the autobiographical memory, which consists of episodes collected from one's own life [Kumar and Clark, 2005]. There is also a progressive loss in a patients ability to learn new information as well as to be able to retain and process it [Kumar and Clark, 2005]. This memory loss presents with a temporal gradient. There is a relative preservation of distant events, but a loss of more recent memory [Kumar and Clark, 2005]. AD is distinguishable from short term memory loss, however, as working memory is preserved [Kumar and Clark, 2005]. The observation of language impairment, apraxia, agnosia and frontal executive function are also tell tale signs of AD [Kumar and Clark, 2005].

#### **Medical Imaging**

An MRI scan can provide more information on the patterns of regional brain atrophy. This is used to diagnose the different types of dementia, for example AD or frontotemporal dementia. In AD, hippocampal atrophy is observed, whereas in frontotemporal dementia, temporal lobe and frontal atrophy is evident [Kumar and Clark, 2005]. In the early stages of AD, a collection of dilated presynaptic neuronal process occur around the amyloid plaque [Cross and Underwood, 2013]. As the plaque matures, it develops an amyloid core, eventually causing what is described as a burnt

out plaque which results in the death of the surrounding matter [Cross and Underwood, 2013]. The effect of this, and the dilation of the ventricles can be seen in Figure 2.2, taken from [Cross and Underwood, 2013]. Patients with MCI under the criteria given by [Petersen et al., 1997] often show intermediate stages of hippocampal atrophy compared to NC and AD [Smith, 2013].



Figure 2.2: A brain demonstrating AD, showing cortical atrophy- white matter loss and dilated ventricles. [Cross and Underwood, 2013]

$A\beta$  can also be deposited in the walls of small arteries and capillaries in the brain, which weakens their walls and can cause ruptures and haemorrhaging [Cross and Underwood, 2013]. Tangles are mainly found in the hippocampus, cerebral cortex, subcortical grey matter and brain stem nuclei [Cross and Underwood, 2013].

### 2.1.4 Treatment

AD is untreatable and there is no current cure [Naish et al., 2009, Kumar and Clark, 2005]. The dementia that it causes is treated using therapeutic strategies with the intention of preserving dignity and keeping patients in a familiar, home environment for as long as possible [Kumar and Clark, 2005]. The burden often falls on relatives [Kumar and Clark, 2005]. There is evidence that a heterogeneous (diverse) psychological group intervention, as well as the prescription of piribedil and donepezil prevent the progression of MCI into AD, however investigations into the treatment of MCI are ongoing [Cooper et al., 2013].

## 2.2 Machine Learning (ML)

ML is a way of designing algorithms by combining ideas from statistics, computer science and other areas that utilise data to solve a problem [Crane, 2019]. In traditional programming, the algorithm written is a series of steps or instructions to be performed to solve a problem [Alpaydin, 2010]. ML fills the gap that faced when these steps are unknown. The data is given, and the form of the output and the problem are known, but the steps for getting from this input data to the output are undefinable by human design [Alpaydin, 2010]. Within this project, for example, the brain scans observed capture all the data that could possibly be needed to identify MCI or AD. But to define a computational algorithm to consistently perform this would be impossible.

In ML programming, frameworks (models) are written that are flexible, and have parameters (weights) that are changed based upon the data itself [Alpaydin, 2010]. Models are *trained* on the data to modify the parameters within the model resulting in fully trained model with set parameters that can solve the problem for unseen data.

### 2.2.1 Machine Learning Pipeline

The ML pipeline describes the steps and stages involved in defining, creating and training a ML model to solve a problem. The areas of focus for the programmer are:

- Data - What data should be fed into the model? How can its quality be ensured? What Preprocessing should be performed upon it? How much data should be used for training, for validation or in the test set?
- Model - What style of ML model best suits the problem? What kind of model should be utilised?
- Hyperparameter tuning - How should the model be set up? What is the best architecture and feature set up in order to give the model the best change of learning during training?
- Evaluation - What observations can be made to evaluate how well the model meets the initial problem? Is its performance desirable? Does it train on the data- how does this effect its performance on unseen data?

These questions define the fundamental stages of the ML pipeline used for this project. Within this section (2.2), the main concepts involved in each of the stages defined above are given, and the remainder of the report presents the steps and the actions that were taken in each stage for the creation of the model to classify AD MCI and NC from brain scans.

## 2.2.2 Classification and Supervised Learning

For ML tasks, the problem determines how the model should handle and create output from the data. This project is best defined as a classification problem. The model must classify whether a brain scan demonstrates AD, MCI or NC. This is a form of supervised learning - the most common form of ML used in practice [Murphy, 2012]. It describes the case when labelled data is used to construct a model. Here, scans labelled as AD, MCI or NC are used to train the model.

## 2.2.3 Data

ML uses data to solve a given problem by modifying the model parameters. Hence, the data has a huge impact on the potential value and quality of the model created using that data - it is very much a case of ‘garbage in, garbage out’ - data that fails to contain relevant information will fail to create a model that can solve the problem [Hunter et al., 2018]. A few important concepts surrounding data are described below.

### The Data Set Size Problem

The amount of skill required in creating a model decreases with the amount of data available- the more data, the easier it is for any model to learn the patterns [Goodfellow, 2016]. According to Goodfellow [2016], ‘a good rule of thumb is that a deep learning algorithm will generally achieve acceptable performance with around 5,000 labelled examples per category’. In medical applications, this causes a problem as access to medical data not only has ethical implications, but is also scarce and, commonly, not digital or centralised [Morik, 2010]. As a result, building a model that tackles and finds solutions for a limited dataset is one of the problems the project faced.

### The Curse of Dimensionality

The curse of dimensionality, a phrase coined by Bellman [2003], applies to many areas, but here refers to the problem that the number of possible configurations of the dataset increases exponentially as the number of variables increases [Goodfellow, 2016]. This is a significant problem when working with three dimensional brain MRI scans. The possible combinations of values within a scan is huge, with each individual pixel holding a value between 0 and 255. Methods to reduce the dimensionality can be taken in the preprocessing of the data.

### Validation Sets

The project data must be divided into three groups- training, testing and validation sets. More detail about how a neural network trains is given in training section of the model segment below, but the training data is the subset used to set the model weights in a forward pass of the training data through the network. The validation set is used to calculate the error between the true labels

and the predictions the model makes and modify the weights through backpropagation. These are all described in more detail below. The test set is then used as an unseen subset of the data that can be used for the evaluation of the final model [Goodfellow, 2016].

## Data Augmentation

Data Augmentation is a method of increasing the size of an image dataset by augmenting the original images [Perez and Wang, 2017]. This is especially useful when training a model using a constricted data set as it means the model has more examples from which to learn the patterns, and varying the data through data augmentation like cropping, rotating, and flipping can reduce the generalisation error within a model, helping it to learn the patterns and not the data [Perez and Wang, 2017].

### 2.2.4 Model

The problem determined a classification model, but there are a number of different classification models to choose from. The dataset consisted of images, which presents a heavy predisposition to employ a Convolutional Neural Network (CNN) because of their affinity for recognising complex and non linear patterns within images. An ensemble of three CNNs was used in order to allow each model to specialise on each of the physical planes within the three dimensional scans. The theory of each aspect of this is given below.

## Neural Networks

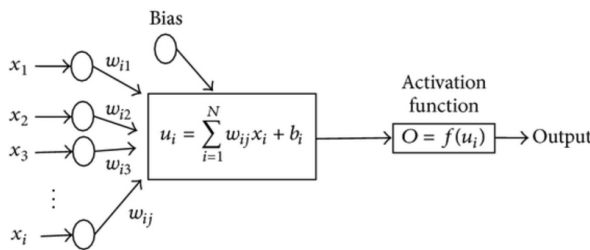


Figure 2.3: The Structure of an Artificial Neuron [Oduncuoglu and Kurt, 2015]

A neural network is a ML method whose structure takes inspiration from the human brain [Hopfield, 1982]. The brain can be abstracted as a network of neurons that are densely connected by synapses, this translates as a graph with neurons as nodes and synapses as edges. Each neuron can be considered as a function that transforms the input from the input edges into an output on the output edge [Anzai, 1992]. The idea of an artificial neuron was first proposed by McCulloch and Pitts [1943]. The McCulloch-Pitts neuron performs

a weighted sum of all the inputs from the edges with an additional bias term, and transforms the result using an activation function into an output. These weights, placed on each of the edges are what the model changes and optimises to "learn". A visual and mathematical summary of this is given in Figure 2.3: The Structure of An Artificial Neuron.

A neural network is composed of layers of multiple neurons, each layer connected to the next, that transforms data from an input layer into an output [Hopfield, 1982]. The number of input nodes is determined by the number of input features, and the number of output nodes is determined by the desired output of the problem. In the proposed problem, there are three output nodes - one for each class (AD,MCI,NC) and the number of input nodes would be  $182 \times 182 \times 182$  if the entire three dimensional MRI scan was taken as input, but this was not the case.

## Training

Neural networks train by adapting the parameters within each neuron (the weights on the edges) based on the training data it experiences. Within supervised learning, this is done by completing a forward pass of the training data through the network and then comparing the label generated by the model with the true label of the data. An error (or loss) function computes the difference between the output of the model and the true label and then an optimiser algorithm, such as stochastic gradient descent, backpropagates through the network, updating each edge weight based on the error [Cheng and Titterington, 1994].

**Loss Functions** The loss (/error/cost) function determines the difference between the output from the model performed on a training sample to its true label - here NC, MCI or AD. It is what the optimiser aims to minimise. Training samples are fed forwards through the network, the loss is calculated, and the optimiser performs back propagation to update the weights of the network based on the loss value. There are different ways of computing the loss function. Cross entropy is the default loss function for multi-class problems. This applies the following formula:

$$\sum_x p(x) * \log(q(x))$$

Where  $p$  is the target distribution - the labels, and  $q$  is the approximation of the target distribution - the model. Sum over all the data samples.

**Optimisers** After performing a forward pass of data through the network, and computing the value of the loss function, a process of backpropagation takes place. Here, the gradient of the loss function with respect to each weight (the parameters the network optimises, in the neurons) is calculated. This allows the weights to be changed and optimised layer by layer (utilising chain rule) through the network to allow it to learn and improve based on the loss [Rumelhart et al., 1986].

The functions that perform these operations are called optimisers, as they optimise the model for the learning task. The most well known method for this is stochastic gradient descent, however an extension to this is an adaptive learning rate method called Adam, which also includes bias correction term [Kingma and Ba, 2014]. The learning rate is the step size by which the weights

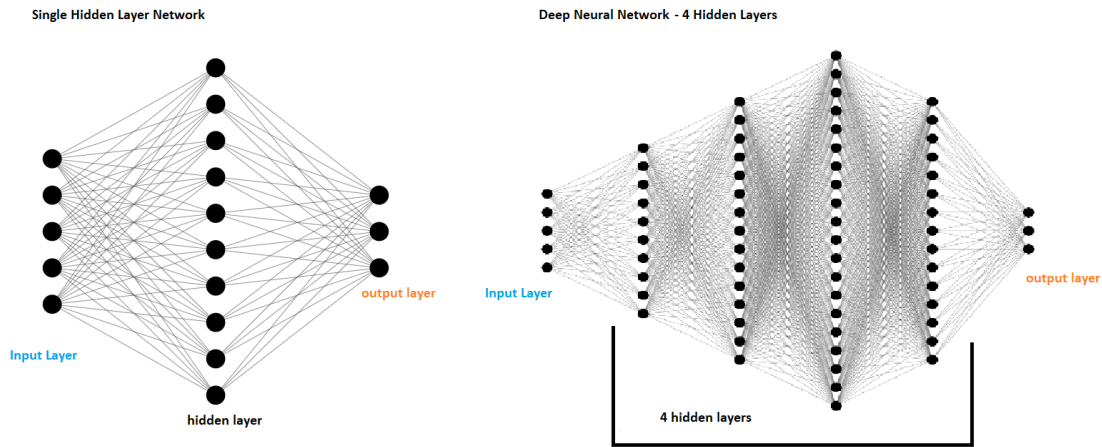


Figure 2.4: A Single Hidden Layer Neural Network and a Deep Neural Network with 4 hidden layers

are updated and is an additional hyperparameter to be determined. The Adam algorithm is given in Appendix B.

## Deep Learning

Deep Learning goes beyond the neuroscientific approach to ML by increasing the number of levels in the construction of a model [Goodfellow, 2016]. A simple neural network has a single hidden layer before the input and output neurons. A Deep Neural Network (DNN) has more than one hidden layer. An example of the structure is given above in Figure 2.4: A Single Layered and Deep Neural Network.

Increasing the number of layers in a network captures a composition of more and more non linear transformations to help the model identify and pick up on the more abstract, and useful patterns within the data [Bengio et al., 2013].

## Convolutional Neural Network (CNN)

A CNN is a form of DNN that uses convolution kernels to perform feature extraction from data [Murphy, 2012]. The weights, rather than being on every edge, are ‘tied’ to the kernel and a convolution describes the linear operation of multiplying the weights of the kernel with the pixel values of a correlating area within the image. The kernel is convolved over all pixels of the image to produce a series of nodes that make up a feature map [Brownlee, 2019]. A visual of this is given to the right of page 21 in Figure 2.5: Operation of a Kernel. Tying the number of weights to the kernel reduces the number of parameters in the network and allows a feature within an image to be

detected no matter where it is positioned [Ganesh, 2019].

An entire convolution layer learns multiple features in parallel, so each layer will learn between 32-512 filters for an input within one layer. The output of the convolution layer as a whole consists of multiple feature maps, each presenting a different aspect learnt from the image [Brownlee, 2019]. The idea is that simpler features are extracted at the surface layers, with more and more detailed, complex features extracted through deeper and deeper convolution layers [Simard et al., 2003].

Another important concept within CNNs is the pooling layer. The general structure of a CNN consists of groupings of three layers: convolutional layer, where the weights of the kernel are applied to produce a feature map, the activation function as described within neural networks, and then a pooling layer [Goodfellow, 2016]. This can be known as the detector phase; it helps to make the output invariant to any small changes in the position of features in the input and further enforces the benefits of CNNs in image processing [Goodfellow, 2016].

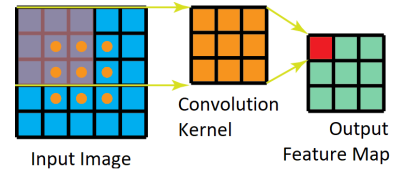


Figure 2.5: Operation of a Kernel on an Input to Produce an Output Feature Map [Ganesh, 2019]

## Ensemble Models

Ensemble Learning describes the process of using multiple ML models to contribute to the final outcome; each model has ‘vote’ towards the overall result [Murphy, 2012]. Different models have different flaws and make different errors, so by creating and training a number of models and ensembling them, the result of the errors of each is minimised [Goodfellow, 2016]. Usually, a number of models are trained on the same data and combined, but for this project a different method was devised. The properties of the data was exploited and a different physical plane of the scan was used as input to each model of the ensemble. The result of this was three models, each trained and specialised on a different axis.

### 2.2.5 Hyperparameters

The model parameters (the weights) are set during training, but there are aspects of the model that must be determined by the programmer before the model is trained. These are the user set hyperparameters. The no free lunch theorem makes clear that the ‘perfect’ ML model is unachievable [Goodfellow, 2016], however by tuning the hyperparameters, an attempt is made to get as close as possible. There are two sets of hyperparameters: the hyperparameters of the network topology, that is the architecture and structure of the model and the hyperparameters of the training, that is hyperparameters that modify how the training is set up to run. The hyperparameters tuned within the project were:

- Model Hyperparameters

- Network Size: This includes network depth, the number of filters and the size of the kernel
- Layer Types: This includes the use of dropout and batch normalisation layers.
- Activation Functions
- Training Hyperparameters
  - Number of Epochs
  - Early Stopping
  - Learning Rate and Batch Size

Each of these is defined and described subsequently.

## Model Hyperparameters

**Network Size** As computational capability has grown, so have the sizes of the networks that can be produced. Larger networks achieve higher accuracy and can handle more difficult and complex problems [Goodfellow, 2016]. When designing the architecture, or topology of a CNN, the number of filters and the number of layers both affect the eventual size of the network. In the case of the CNN, this is also effected by the size of the kernel used, whether padding is used to include the edges of the images, and whether a pooling layer is used, as well as what stride this takes. The larger the network, the more parameters it will need to train, and using an oversized network to solve the problem risks overfitting (described in evaluating training).

**Layer Types** Besides convolution and pooling layers, there are additional options such as batch normalisation and dropout. Batch normalisation was proposed by Ioffe and Szegedy [2015] to make neural networks faster and more stable. Dropout layers are a regularisation method (aimed at reducing overfitting) that randomly omit units when training the network [Hinton et al., 2012].

**Activation Functions** The activation function transforms the sum of the weighted inputs to a neuron to produce the output of that neuron. Five different activation functions are given below in Figure 2.6: Example Activation Functions. ReLu is the most popular and widely used within deep learning domains, however it is saturated towards zero which can create a problem in backpropagation by causing the gradient to become zero and the model not to learn [Pareek, 2020]. Leaky ReLU and EReLU seek to avoid this.

## Training Hyperparameters

**Number of Epochs** A single epoch is one complete forward pass of the training data through the network. The number of epochs describes the number of forward passes of the complete training dataset through the network used for training a model/network.

**Early Stopping** Early stopping is a technique whereby the value of a chosen metric, such as loss or accuracy, is observed during training and a patience value is defined such that training will be halted if the chosen metric demonstrates a change beyond the selected patience [Prechelt, 1998]. For example, if validation loss begins to increase by more than 0.1, it is a sign of overfitting as the model is ceasing to improve in its performance on the validation set and get further away from the true values.

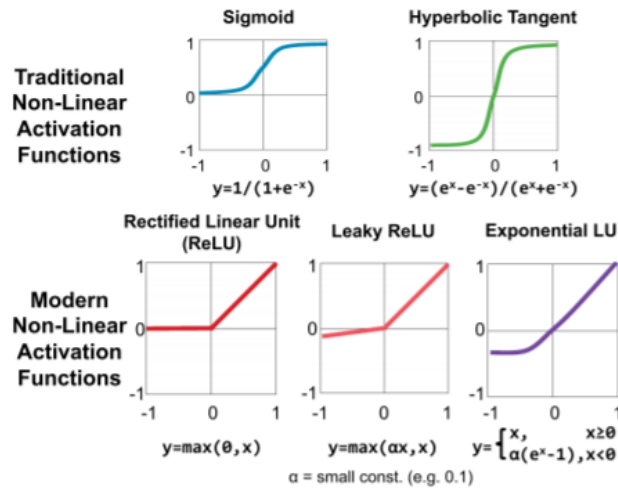


Figure 2.6: Example Activation Functions [Sze et al., 2017]

## Learning Rate and Batch Size

The learning rate of a model defines how much the model weights should be updated after each epoch and the batch size determines how many samples should be used within a single pass forward/back through the network [Kandel and Castelli, 2020]. These two are related as more data should warrant a larger change in weights. A smaller batch size finds better minimums in the sense of validation [Keskar et al., 2016] and should be paired with a smaller learning rate. A batch size of 1 would denote a stochastic gradient descent optimisation- a single data item is fed forwards through the network and the weights are updated based on the learning rate in backpropagation.

## 2.2.6 Evaluation

### Evaluating Training - Under and Overfitting

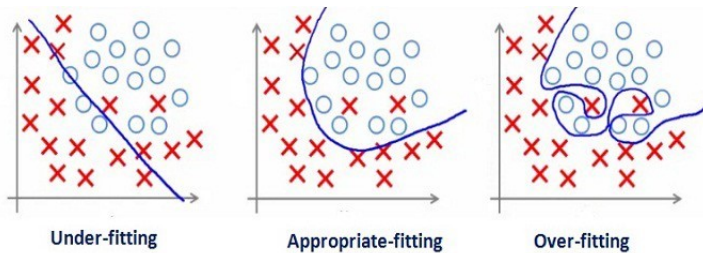


Figure 2.7: Under and Over Fitting on a Binary Classification Problem [Patel, 2018]

the model is too simple to sufficiently identify patterns in the data to solve the problem; here both

When the model is trained, there are two different losses to consider, first the training error (the difference between the labels and generated labels for the training set), but a more telling method of performance is the testing error (also called generalisation error). Methods such as the use of a validation set, and dropout layers help to optimise the generalisation error. Underfitting occurs when

training and testing error would be high. Overfitting occurs when the data learns the training set too well and is unable to generalise this to information it hasn't seen; here the training error would be low, but the testing error high. This can be seen in a simple binary classification model in Figure 2.7: Under and over Fitting beneath the subsection title. It is easy to identify both cases - in underfitting, the model simply will not achieve the performance metrics desired. In overfitting, the validation error begins to increase as the model becomes over-trained and over-focused on the training data. In addition to the measures to reduce the generalisation error, the early stopping of training can be used to prevent this.

### Evaluating Performance - Metrics

| Metric                           | Formula                                                                                                                 |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Accuracy                         | $\frac{\text{Number of correct predictions}}{\text{total predictions}}$                                                 |
| Loss (Error)                     | Dependent on loss function used - the difference between the classification values and the true values- lower is better |
| Sensitivity (true positive rate) | $\frac{tp}{fn+tp}$                                                                                                      |
| Specificity (True Negative Rate) | $\frac{tn}{tn+fp}$                                                                                                      |
| False Positive Rate              | $\frac{fp}{fp/(fp+tn)}$                                                                                                 |
| Area under curve                 | Area underneath a curve plotted (false positive rate $\times$ true positive rate)                                       |
| Precision                        | $\frac{tp}{tp+fp}$                                                                                                      |
| Recall                           | $\frac{tp}{tp+fn}$                                                                                                      |
| F1 Score                         | $2 \times \frac{1}{\frac{1}{precision} + \frac{1}{recall}}$                                                             |

Table 2.1: Performance Metrics for a Machine Learning Model.

There are a number of metrics that can be observed to evaluate the success of a model. These include standard observations, such as accuracy scores and looking at the loss (error) values. However, for most problems a lot more information can be gained by delving deeper into the performance of a model. Especially within a classification problem such as this, using a confusion matrix, which displays the true label and the classified labels of a sample, can demonstrate whether a model has learnt how to classify one class better than the others, or whether there is a particular class it struggles with. This is vital for the proposed project, as identifying the MCI stage can allow preventative measures to prevent the progression to AD. Table 2.1 above shows how many of these metrics are calculated, with fp, fn, tn and tp meaning false positive, false negative, true negative and true positive. Because this is a multiclass problem, the fp/fn/tn/tp rates apply for each class and are combined.

## **K Fold Cross Validation**

Cross validation attempts to get a true measure of the performance of a model by performing various rounds of training and testing on several splits of the data [Arlot and Celisse, 2010]. The hold out method has just a single test set, the K fold cross validation method creates  $k$  splits of the data, testing on one set and training on the remainder each time, and the leave one out method is a more exhaustive method of  $k$ -fold cross validation, such that  $k = \text{number of samples}$  - that is, one data sample is left out each time [Arlot and Celisse, 2010]. As previously, the validation data set is used to reduce the test error and is made up of a sub-sample of the training set. A stratified approach ensures an even distribution of the class labels of data within each set [Arlot and Celisse, 2010].

# Chapter 3

## Previous Work

### 3.1 Literary Review

The review of previous work was divided into two sections. Firstly, what has been used as input to ML models that can perform similar classifications of brain scans and secondly, how have advances in ML improved and developed methods that can be applied to this classification problem. A summary of the different studies that have been investigated and analysed is given in Table 3.1 on page 32. There are a large number of studies that have been conducted and are applicable within this area, but the most relevant and recent progress have been chosen for analysis.

#### 3.1.1 Preprocessing - Model Input

Section 2.2.3 (Data) demonstrated how the data that is input into an ML model determines its success. It is vital that the data should be succinct, in that each input contains all the necessary and valuable information required to perform the classification and nothing else.

Each of the studies that have been investigated have used MRI scans as in this project. MRI scans present a structural approach to performing the classification, based on the atrophic process of AD in brain regions [Rathore et al., 2017]. There are three main methods of feature extraction of these voxel based images: density maps, cortical, and predefined region based methods [Rathore et al., 2017]. The main focus was on region based methods, since the development in the research of the atrophy of AD means key regions of the brain can be identified that exhibit features of the disease.

#### Dimensionality

The data used as input to the models in the reviewed work vary considerably, from using the 3D scans themselves [Payan and Montana, 2015, Wang et al., 2018] to transforming the data into a single dimension of features [Gupta et al., 2019, Choi et al., 2018].

The ML model chosen also has an impact here. This project utilised a CNN which can be applied to two or three dimensional data, so some of the preprocessing demonstrated by some of the studies, such as feature extraction and flattening, can be eliminated because of the use of a CNN. Gupta et al. [2019] employed random tree embedding to map the data to a high dimensionality state and singular value decomposition to reduce the dimensionality [Gupta et al., 2019]. Choi et al. [2018] trained a DNN to learn the relationships between the voxels of the scan and to output the regional abnormality for each brain region, then the classifier used these values as the input to perform the classification.

To the opposite extent, some studies used the full three dimensional scans to retain the structure and patterns within the data. Payan and Montana [2015] Compared both 2D and 3D CNNs, using sparse autoencoder to first learn the filter weights for the convolution layers. The 3D CNN used the  $68 \times 95 \times 79$  dimensional MRI scans as the input and consisted of a convolutional layer pretrained using the sparse autoencoder, followed by a pooling layer, then a fully connected layer, and a final 3 class output (NC, MCI, AD). In the 2D approach, slices were extracted of size  $11 \times 11$  and 68 slices were used as the input for each scan. The 3D CNN either met or exceeded the performance of the 2D CNN in all cases. It showed the smallest increase in effectiveness when classifying AD vs NC and NC vs MCI. The two dimensional slice route has been adopted for this project due to the vast improvements in computation time, as well as the small dataset size and the curse of dimensionality.

### Regions of Interest (ROI)

The limbic system (entorhinal cortex and hippocampus) show the first signs of MCI [Cross and Underwood, 2013]. However, Gupta et al. [2019] demonstrated that including multiple biomarkers can improve the classification accuracy of a model. In their study, 246 separate ROI were used, along with 246 features consisting of the quantity of GM within each region. 210 of these were cortical and 36 sub-cortical [Gupta et al., 2019]. These regions are shown in Figure 3.1 on the right.

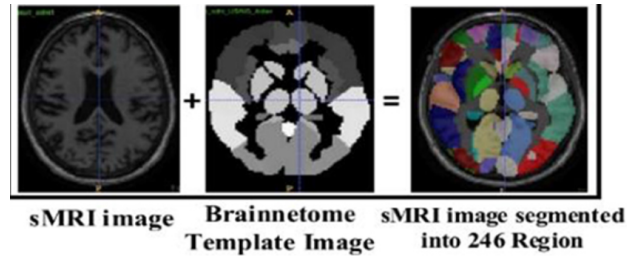


Figure 3.1: 246 Regions used as input for Gupta et al. [2019]

Ma et al. [2016] found that AD patients displayed Grey Matter Volume (GMV) loss in the hippocampus and entorhinal cortex. The MCI patient showed GMV decline in the bilateral hippocampus and the right entorhinal cortex but with smaller area of decline and only slight cortical thinning. In the NC, slight cortical thinning could be observed but no decrease in the GMV.

Choi et al. [2018] used 93 regions and acquired regional volumetric maps for each. For each region, random subspaces were constructed, defined by randomly selecting voxels within that region.

## Skull Stripping and Linear Registration

Using MRI scans within a ML model requires certain preprocessing: the images must be skull stripped, linearly registered and segmented before the regions of interest can be extracted. Skull stripping describes the process of removing the bone and skull from the scan. Linear registration is the process of transforming medical images so that a template can be applied; it ensures that the possible regions of interest have the same coordinates across all scans. Segmentation describes the process of separating the Grey Matter from the white matter and the cerebrospinal spinal fluid. The ROI can then be extracted using a template.

In Gupta et al. [2019], the processing of the MRI scans consisted of anterior commissure–posterior commissure correction and N4 bias field correction. These techniques were performed to correct the intensity of inhomogeneity for each image. They were aligned using the MNI152 T1-weighted standard image before performing the region extraction. Ma et al. [2016] used SPM8 and DARTEL in correcting, unifying and segmenting the scans and creating the templates to register the scans. Payan and Montana [2015] used Statistical Parametric Mapping to normalise the images to a brain mapping template. Choi et al. [2018] performed a process of anterior commissure–posterior commissure correction, skull stripping and cerebellum removal. The intensity and inhomogeneity were corrected and the scans were segmented into Grey Matter, white matter and cerebrospinal fluid using FSL, with the Grey Matter images being used as the final model input.

## Extraction of 2D Planes

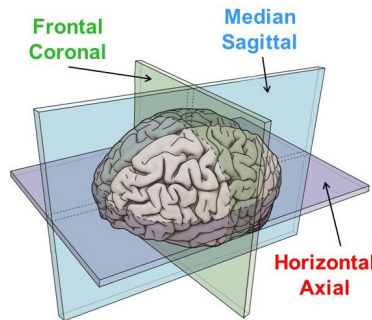


Figure 3.2: Anatomical Planes. Purves et al. [2012]

Islam and Zhang [2018] created three ‘patches’ for each scan based on the axial, sagittal and coronal planes, as given in Figure 3.2:Anatomical Planes. By performing this slicing, the 3D scan is separated into three 2D images, but the dimensional data is retained within the grouping of these three images. This approach has greatly influenced the approach of this project, where the ROI map is separated into three images based on these planes. This has also been taken a step further, noting the importance and the depth of detail within multiple biomarkers as demonstrated by Gupta et al. [2019], each plane is used as input to its own separate model.

## **Data Augmentation**

Islam and Zhang [2018] and Wang et al. [2018] demonstrated methods for the data augmentation of brain MRI images. Wang et al. [2018] began with 196 Images. Rotation, gamma correction, noise injection, random translation, and scaling created 180 new images for each of the original images, resulting in a dataset of 17,738 images.

### **3.1.2 Classification Methods**

Historically, the classification of brain scans into NC or AD would have been performed by an expert Doctor or Physician. This progressed through learning the atrophy of AD to narrowing down the ROI within the brain. The initial ML approach to the problem was to use methods such as support vector machines (described below). With the development of computation to allow the use of more complex learning models, the field developed into the use of deep, convolutional methods to identify even more delicate classifications, such as that of MCI.

#### **Support Vector Machines**

A few of the studies observed utilised support vector machines. A support vector machine is a linear ML model that works by establishing a best fit hyperplane that separates two binary groups of data and can also be extended to perform multi-class classification. Non linear methods, such as CNNs have demonstrated a performance increase compared to support vector machines [Sudha et al., 2019].

#### **Convolutional Neural Networks**

Islam and Zhang [2018], Payan and Montana [2015], Wang et al. [2018], Choi et al. [2018] all utilised CNNs for various input methods of the brain scans. CNNs have been proven to outperform standard linear techniques of both multivariate and nearest-neighbour discriminant analysis, as well as z statistics [Pritchard et al., 1994, Anderer et al., 1994]. A two dimensional CNN has already been established for this project by observing Payan and Montana [2015] slice extraction methods. The question that remains is the depth of the network and the layers utilised within it.

Numerous structures, depths, layer types, loss functions, and activation functions have been explored throughout the literature but these have not lead to a definitive architecture, they are all different and similarly successful. So these hyperparameters will be set based upon experimentation rather than the literature.

#### **Independent Test Set**

Abdulkadir et al. [2014] used a completely independent test set from the AIBL database, on a model trained using ADNI data. This demonstrated that models trained on data from one source

can still be applied to MRI scan data from other sources. This inspires confidence that the model created by the project, although trained on the ADNI dataset, will perform similarly given a new MRI scan from a different machine.

## Model Output

A common problem occurs when comparing previous literature to influence this project. The model outputs, i.e. the NC, MCI and AD states of the brain that which this project classified, is done differently within other studies. Gupta et al. [2019], Islam and Zhang [2018] and Choi et al. [2018] approached the slightly different problem of classifying the data into both mild MCI (MCIs) and progressive MCI (MCIp) stages, creating a four class classification problem. Wang et al. [2018] approached just the NC/AD problem. This does not impact the comparison between the work too greatly, as the studies looked for the same signs within the brain scans. What is interesting is how some work such as Choi et al. [2018] chose to break the multiclass classification problem into binary ones and train separate classifiers for each stage of the disease. However, this didn't appear to demonstrate a huge improvement compared to the accuracies of the work that approached the multiclass classification problem. Hence, the approach taken by this project was to retain the problem brief and to create a single system for the classification of the three categories.

| Summary of classification of AD Approaches |                                           |                                                                                                                                                                                                                                                  |                                                                                                                                                               |                                                                      |
|--------------------------------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Reference                                  | Data                                      | Model Input                                                                                                                                                                                                                                      | Model Type                                                                                                                                                    | Accuracy                                                             |
| Gupta et al. [2019]                        | ADNI<br>38 AD 36<br>MCIs 46 MCIC<br>38 HC | 210 Cortical,<br>36 sub Cortical<br>regions (Trans-<br>formed using<br>random tree<br>embedding and<br>Singular Value<br>Composition)<br>+ volume of<br>grey matter in<br>each region<br>+ 3 biomarker<br>features and<br>2 genotype<br>features | SVM<br>Six binary clas-<br>sification prob-<br>lems<br>m (AD vs.<br>HC, MCIC vs.<br>MCIs, AD vs.<br>MCIC, HC vs.<br>MCIs, HC vs.<br>MCIC, and AD<br>vs. MCIs) | Range from<br>92% - 98%<br>Based on the<br>classification<br>problem |

|                          |                                                                                                                          |                                                                                                                                                |                                                                                                           |                                                                                                   |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Islam and Zhang [2018]   | OASIS (Open access series of imaging studies)<br>416 Data Samples<br>Data augmentation to increase the number of samples | Patches based on the three physical planes with dimension 112*112                                                                              | Deep CNN using Convolution, Batch Normalisation, ReLu activation and Pooling layers. SGD as the optimiser | 99% for NC, 75% for Very Mild AD, 62% for Mild AD and 33% for Moderate AD.                        |
| Payan and Montana [2015] | ADNI<br>2265 Historical Scans                                                                                            | 3D method -<br>The 3D MRI scans<br>Each image had the dimensions $68 \times 95 \times 79$ .<br>2D method - 68<br>2D slices taken from the scan | 3D CNN                                                                                                    | 3D - 89.47%<br>2D - 85.53%<br>Worst performance for both observed in the AD vs MCI classification |
| Wang et al. [2018]       | OASIS<br>196 Images<br>Data augmentation resulted in a dataset of 17,738 images.<br>Half AD, Half NC                     | 3D Scans                                                                                                                                       | Eight Layer CNN with Leaky Relu and Max Pooling and SGDM (stochastic gradient descent with momentum)      | 97.65% (AD vs NC binary classification)                                                           |

|                                |                                                                                                                                              |                                                                                                                                                                                                                                                                                |                         |                                                                                                               |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------|
| Ma et al.<br>[2016]            | ADNI<br>15 AD, 23<br>MCI, 51 NC                                                                                                              | Correct and<br>unify tissue<br>class images<br>using SPM8<br>and DARTEL,<br>registered to<br>Dartel tem-<br>plates. Gray<br>matter images<br>normalised to<br>the MNI space.<br>Six sub re-<br>gions - bilateral<br>hippocampus,<br>amygdala and<br>parahippocam-<br>pal gyrus | Z Score maps<br>and SVM | 74.6% for the 3<br>class classifica-<br>tion problem<br>92.7% for NC,<br>91.7% for AD<br>and 78.4% for<br>MCI |
| Abdulkadir<br>et al.<br>[2014] | ADNI and<br>AIBL<br>1429 samples                                                                                                             | Grey Matter<br>Intensity Maps                                                                                                                                                                                                                                                  | SVM                     | 41.7% (HC vs<br>rest)<br>66.7% (MCI vs<br>rest)<br>77.8% (AD vs<br>rest)                                      |
| Choi et al.<br>[2018]          | ADNI<br>801 subjects<br>229 NC, 374<br>MCI, 198 AD<br>MCI divided<br>into 214 stable<br>MCI (MCIs)<br>and 160 pro-<br>gressive MCI<br>(MCIp) | The output of<br>a DNN of the<br>probability of<br>regional abnor-<br>mality for each<br>region                                                                                                                                                                                | Three Layer<br>NN       | 89.22% (MCI<br>vs CN)<br>88.52% (MCIp<br>vs MCIs)                                                             |

Table 3.1: A Summary of past work in the classification of AD

## 3.2 Implementation of Previous Methods

This project builds upon the preprocessing methods devised by Dai [2019]. This section seeks to set out how these previous methods have been utilised and improved upon.

### 3.2.1 Data - Ethics

”Data used in the preparation of this article were obtained from the Alzheimer’s Disease Neuroimaging Initiative (ADNI <sup>1</sup>) database (adni.loni.usc.edu). The ADNI was launched in 2003 as a public-private partnership, led by Principal Investigator Michael W. Weiner, MD. The primary goal of ADNI has been to test whether serial magnetic resonance imaging (MRI), positron emission tomography (PET), other biological markers, and clinical and neuropsychological assessment can be combined to measure the progression of mild cognitive impairment (MCI) and early Alzheimer’s disease (AD).”

The data contain no personal identifiers and hence there is no risk of revealing personal, confidential or sensitive information that can be connected back to an individual. The author additionally filled a registration with the ADNI initiative allowing the use of the data assuring use of the data for research purposes. The project also underwent approval using the University of Manchester’s ethical approval tool <sup>2</sup> which indicated that the project did not demonstrate a requirement of formal ethical approval through committee.

---

<sup>1</sup>Data used in preparation of this article were obtained from the Alzheimer’s Disease Neuroimaging Initiative (ADNI) database (adni.loni.usc.edu). As such, the investigators within the ADNI contributed to the design and implementation of ADNI and/or provided data but did not participate in analysis or writing of this report. A complete listing of ADNI investigators can be found at: [http://adni.loni.usc.edu/wp-content/uploads/how\\_to\\_apply/ADNI\\_Acknowledgement\\_List.pdf](http://adni.loni.usc.edu/wp-content/uploads/how_to_apply/ADNI_Acknowledgement_List.pdf)

<sup>2</sup>[https://www.training.itservices.manchester.ac.uk/uom/ERM/ethics\\_decision\\_tool/story\\_html5.html](https://www.training.itservices.manchester.ac.uk/uom/ERM/ethics_decision_tool/story_html5.html)

### 3.2.2 Data - Preprocessing

The MRI data processing conducted by Dai [2019] entailed:

#### 1. Skull Stripping

ROBEX was chosen to perform the task of removing the skull, neck and eyes from the MRI scans. This was compared to and chosen over FSL-BET due to retaining more of the brain structure and requiring less hyperparameter tuning [Dai, 2019]

#### 2. Linear Image Registration

Image Registration, in this case, describes the process of conforming the coordinates of the key regions within each brain scan through affine transformations. FMRIB's linear image registration tool (FLIRT) was used to conform the scans with the MNI152 template.

#### 3. Segmentation

FSL-FAST was used to separate out the grey matter after comparison with Statistical Parametric Mapping (SPM) because of difficulties acquiring the MATLAB license required for SPM. The FSL-FAST algorithm utilises hidden Markov random field model and the Expectation-Maximisation of the fluid that is desired. It is completely automatic, is robust and reliable and demonstrates little sensitivity to noise [Dai, 2019].

#### 4. Region of Interest Extraction

The Region of Interest template was established using Harvard-Oxford cortical and subcortical structural atlases. It comprised of seven sub structures: Hippocampus, amygdala, caudate, putamen, pallidum, thalamus and mesial temporal lobe [Dai, 2019]. A threshold value of 0.3 was used to eliminate any voxels with a probability of less than 0.3 from the resulting structure

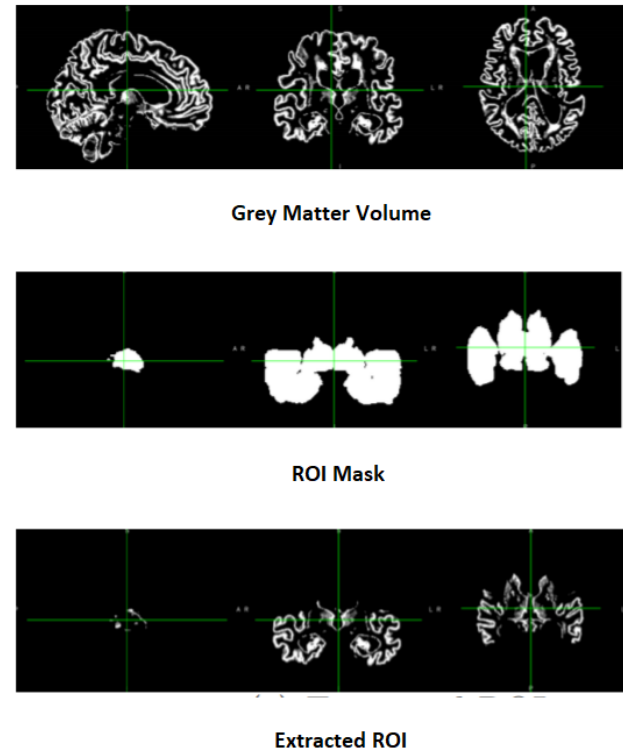


Figure 3.3: Segmentation and Region of Interest Extraction of the three dimensional MRI Scans - Shown as three two dimensional images

Dai [2019]

The output of stages 3 and 4 are given above in Figure 3.3. This shows the three dimensional data as three two dimensional images across the three anatomical planes, but the data itself is three dimensional. Nothing about this process was changed, as the three dimensional MRI scans obtained for the project had already undergone this process.

### 3.2.3 Data - 2.5D Slice Extraction

From the preprocessed three dimensional MRI scans, Dai [2019] devised a method to transform the three dimensional structures into a two dimensional representation that retained the atrophy required for the classification task. The task here for this project was to use and improve upon this. The improvements are given below:

- As the initial improvement performed in this project, this process was parallelised. Previously, each scan was taken and transformed individually. **These improvements took the run time from 367.334 seconds to 77.0417 seconds.**
- Then, a number of areas were identified where the code did not produce the outcome that was intended or desired.
  - The first comes in the 2.5D Slice extraction. The idea here was to take multiple slices across each physical axis and to merge the slices into a single image that incorporated the multiple slices in one colour channel. The result would be a 2.5D image with red green and blue channels, each containing an aggregation of multiple slices from each axis. It was discovered that the operation used to perform this (imwrite) instead overwrote each slice instead of overlaying the images on top of each other. The effect of this is that **instead of having a number of slices from each axis overlaid into one colour channel, the image in fact only contained the single, final slice of the loop from each axis.** The difference in the images is demonstrated below in Figure 3.4. This issue was fixed by converting the image to a 2D array of pixel brightnesses, cumulatively adding each slice and normalising the result, before writing the final result to an image file.

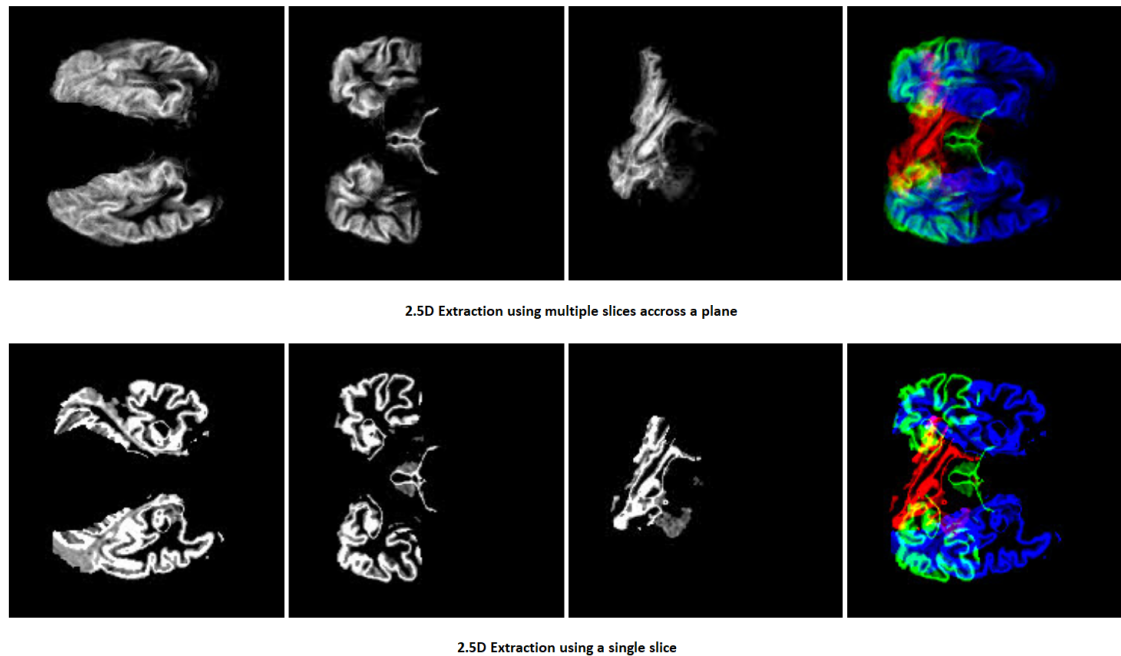


Figure 3.4: The Visual difference when including multiple slices along an axis as opposed to singular slices

- After creating these 2.5D images, a process of flattening the data was performed then saved as a one dimensional array. This was later reshaped when it was read to transform the data back into an array of 2D images. This was a process that was not only unnecessary, as the data can be stored as two dimensional, but upon further inspection it had consequences for the processing. After reshaping, a single brain scan item looked like the image given to the right, in Figure 3.5. The reshape function reassembled the one-dimensional array in the wrong order, having the undesired effect of shuffling the data points to recreate the image as something completely different to the original. **This part of the preprocessing was eliminated and instead the data was saved with the original shape (1070 \* 64 \* 64 \* 3). This saved unnecessary data manipulation and processing as well as eliminating the bug.**

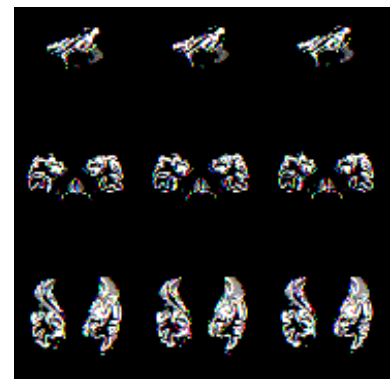


Figure 3.5: Resulting Image of a single scan after saving as 1D array, reading it in and reshaping

- Finally, the data augmentation implemented by [Dai, 2019] was done so during the extraction of the 2.5D slices, **before it had been divided into the train, test and validate sets.** This was completely removed and any data augmentation was implemented solely on the

training set following from the split of the data for this project. The original report also recorded creating multiple data samples from each three dimensional scan by extracting different samples using different centre points from it - again this would mean that samples in the test set could previously have been seen in a different form within the test set. This was not implemented within this project and it remains unclear how Dai [2019] achieved a dataset of 38,521 samples before data augmentation which then increased this to 57,779 samples.

The resulting code for the creation of the 2.5D png images from the MRI scans is given in Appendix A.

# Chapter 4

## Research Methods

### 4.1 Methods

The project was conducted using python version 3.8.5, Tensorflow 2.4.1 and anaconda 4.9.2. Visual Studio Code and Jupyter Notebooks were employed as IDEs. Due to COVID-19, the project was first executed on a personal computer with an Intel i5-7200U CPU. From 2021 onwards, the processing was shifted to an NVIDIA GTX 1060 6GB GPU to spare the CPU of its suffering.

Unless defined otherwise, experiments were performed using k-fold validation where  $k = 10$ , with equal samples from each group (243 from each), a 0.2 split for test data, and a 0.2 split of the training data into the validation set. This resulted in 145 test samples, 116 validation samples, 466 training samples. For the experiments, the input image included all three planes in each of the colour channels, as in Figure 4.1, and had dimensions  $64 \times 64 \times 3$ . A single classifier (not ensemble model) was used. When required, the separate images of each plane were obtained by splitting up the channels of an image such as above to reduce the amount of storage required.

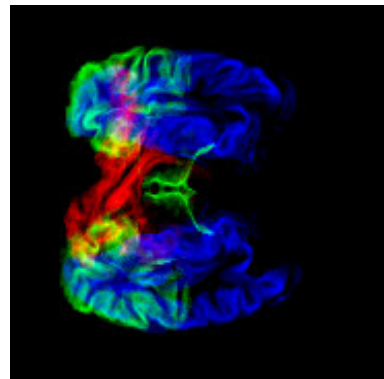


Figure 4.1: Example Model Input Image

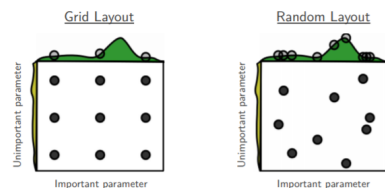


Figure 4.2: Random Search for configuring network hyperparameters Bergstra and Bengio [2012]

### 4.2 Initial Model

The model configuration used for the experiments was established using Keras Tuner employing a random search [Solis and Wets, 1981].

Random search tunes the hyperparameters of a neural network within a fraction of the time it would taken to manually or grid search every possible combination of the hyperparameters, and finds a

configuration as good as or better than this approach [Bergstra and Bengio, 2012]. This is seen above in Figure 4.2. A number of random searches were performed in sets of 20 trials, each time different parameters were changed, but the number of filters was deliberately limited so as not to cause an exponential increase in model parameters that would increase the training time. The chosen model shows comparative performance to larger structures, achieving a training accuracy of 0.88 (2s.f.) and an accuracy of 0.75 (2s.f.) when performed using k-fold validation with  $k = 5$ , yet is much smaller and considerably quicker to train. This not only increased the efficiency of the experiments, but meant that k-fold validation was possible, increasing the reliability of the results too. Its structure can be seen below in Figure 4.3, and the code in appendix C

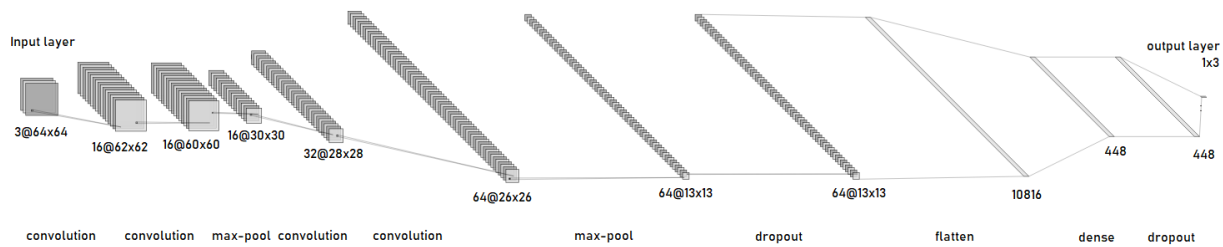


Figure 4.3: Neural Network Structure

## 4.3 Considerations

It must be noted that the experiments each changed a certain factor of the design of the system and all other factors were not changed. It is therefore impossible to say that, having used a different architecture or optimised a model for each variation of the factor, the results could have been very different. This is the nature of a project such as this- there are so many hyperparameters to change and different possible arrangements and variations on the architecture and techniques, that an optimised topology is not achievable. Heuristics such as Occam's Razor help in this process, which says to always aim for the most simplistic solution. The experiments undertaken were done to understand the effects that the factors looked at had. The outcome, then, was not to create the perfect preprocessing or model, but to create an informed solution.

# Chapter 5

## Preprocessing

A number of experiments were performed that created the final method of data preprocessing that generated the data used as input for the final model.

### 5.1 Image Size and Cropping

The first experiment performed was to view the effects of image resizing on the final results. The original MRI scan had dimensions of  $182 \times 182 \times 182$ . After the slices were taken, the images were resized using antialias, a high-quality downsampling filter using PILLOW (PIL). Reducing the size of the input image reduces the amount of time taken to train as well as the dimensionality of the dataset. This experiment was conducted in order to observe whether image size had an impact on performance.

The results, given in Figure 5.1:Image Resize Experiment on page 41, are not as expected. The expected outcome was that the larger the images, the more detailed information would be retained and hence the more patterns and details would be available to aid in the classification of the images. This was a trend up to an image size of 160, however the improvements were not significant enough to suggest that reducing the image size created a dramatically inferior result. Resizing the images to a smaller size produced desirable results- the accuracy wasn't severely affected, the individual class accuracies didn't suffer and a considerable amount of time was gained. An additional consideration was memory requirements. Using smaller images allows for a larger and more complex model, without causing a memory overflow on the GPU.

### 5.2 Slice Point

Part of the preprocessing was to select, extract and merge the 2D images of each physical plane from the MRI scans. Choosing where to centre the plane extraction is vital for ensuring that the input to the model contains exactly the information required to perform the classification. To

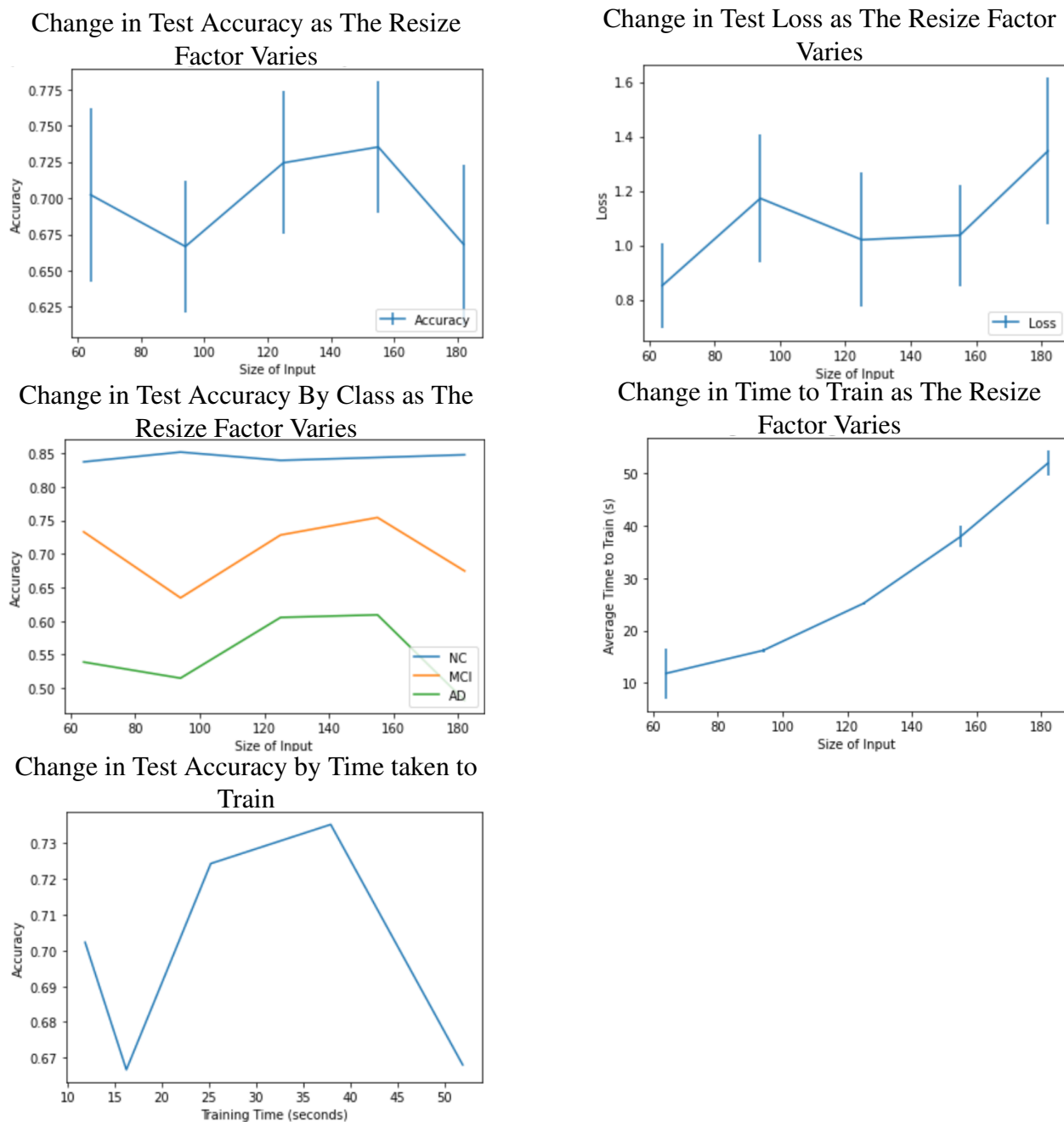


Figure 5.1: Image Resize Experiment

determine where to centre each physical plane, three models were created to take as input a single slice from a plane and to access the features of each slice for the classification. From here, a centre point for the extraction on each plane was determined.



Figure 5.2: Slice Point Experiment

The range of values for each was selected based on Dai [2019] and so as to allow for multiple planes to be incorporated either side. red, green and blue are references to the channel that the result slice is encoded into the image as in Figure 4.1. Red refers to the sagittal plane, green to the coronal plane and blue to the transverse plane. XSLICE, YSLICE, ZSLICE refer to the point upon the relevant axis that the slice was taken from in pixels of the original  $182 \times 182 \times 182$  images, before resizing. From the results, given in Figure 5.2 above, XSLICE was chosen as 72, because

of the clear decrease given by the loss graph. YSLICE was chosen as 118 because of the accuracy, loss and additionally the improvement shown in the MCI class. ZSLICE was chosen as 63 due to the accuracy, loss and again because of the improvement in the MCI classification.

It is also useful to note that it does not appear that any particular plane has an affinity to a certain class, other than AD and NC being generally easier to classify than MCI. The coronal plane (green) has the highest average class accuracy but as a whole all three planes performed similarly for the classification.

### 5.3 Plane Size

The original intention of Dai [2019] was for each of the two dimensional images of each plane to include a number of slices spanning across a selected region of the axis. This plane size experiment looked at how many slices should be incorporated into each plane image. Three models were used- a separate model for each plane that took just the image from that plane as input. This allowed the effect to be determined for each plane individually. A visual of a scan with different plane size amounts is given in Figure 5.3. The dullness that resulted from compounding the slices did not affect the model itself as all values are normalised, and the model is trained relative to each image.

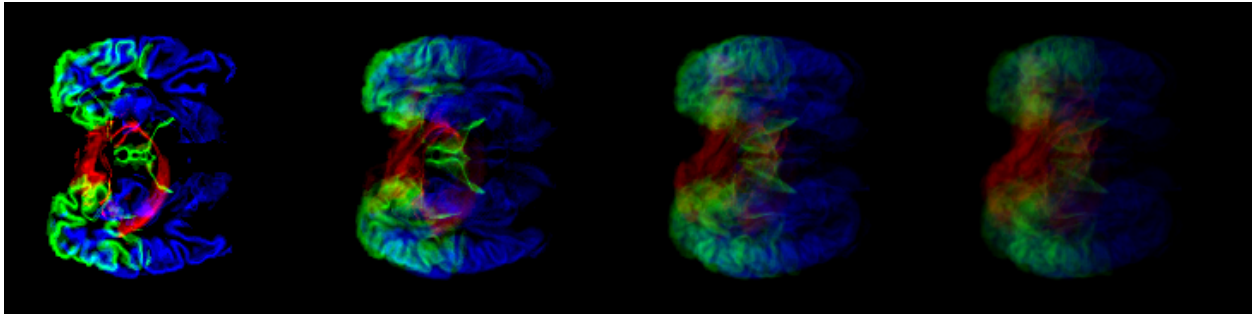


Figure 5.3: Incorporating Multiple Slices from a Single slice to 10 to 30 to 60 ( Left to Right)

The results, shown below in Figure 5.4, demonstrated that the model performed best with a singular slice as the input, which was not expected. It appears that combining the multiple slices along the plane did not increase the detail and information of the 2D image but instead confused and muffled it.

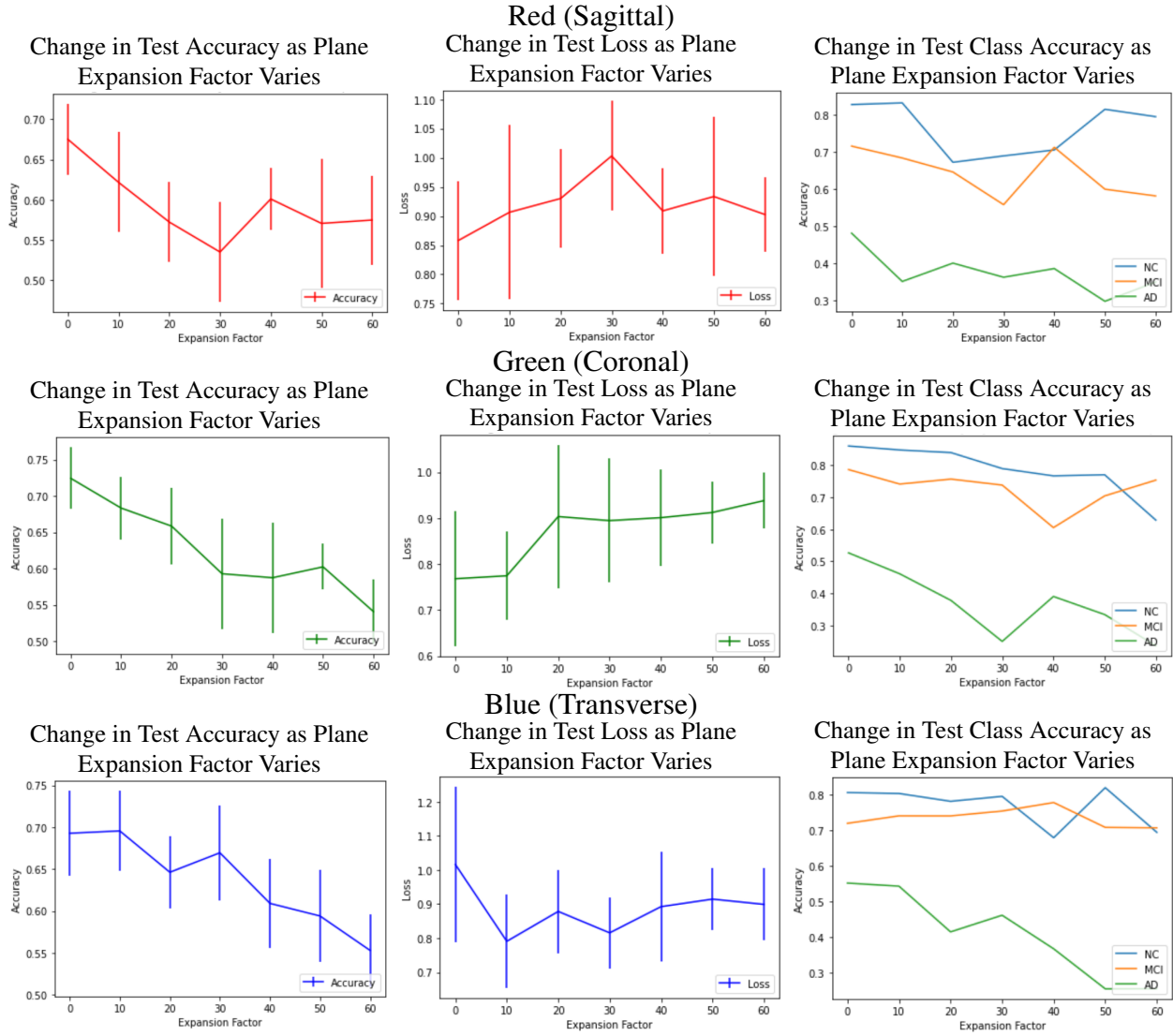


Figure 5.4: Plane Size Experiment

## 5.4 Class Imbalance

The distribution of the classes in the data have an impact on how the model trains and performs. If there is a large sample size in one class, then the model can achieve good accuracy but simply classify everything into this set. For this reason, the data would usually be balanced between the three classes. This is the route that was taken on all the experiments, but it was important to know for the final model whether an imbalance could be utilised to allow the model to learn difficult features, such as those that distinguish the MCI class.

The number of samples from each class in the test set remained equal each time, and the accuracies and losses refer to those observed when testing on this equally split data. The training set (and hence the validation set taken from that) is the set in which the number of samples from each set was varied.

### 5.4.1 Minimising

The first experiments, with results in Figure 5.5, looked at the affect of varying one class at a time to view the result of having a minimised amount of that class. The overall accuracy demonstrated the expected result- the more even the sets, the better the accuracy. What was particularly interesting was the loss and class accuracies when varying the AD set. When varying the number of AD samples, the loss remained high and did not begin to decrease until over 100 samples were seen. This suggests it is particularly hard to distinguish. This pattern continued when looking at the class accuracies. Here, what is particularly interesting to observe is the effect that varying a set had on the classifications of the other two sets. What is clear is that varying the size of the AD set had no effect on the NC and MCI accuracies. There is a slight effect on both AD and MCI when varying AD. A clear correlation is that as the size of the MCI set varied, the class accuracy in classifying the AD samples decreased. Hence, the distinction between NC and AD was easily learnt, but the introduction of the MCI samples made the classification of AD harder.

### 5.4.2 Maximising

The second set of experiments varied two classes to view the affect of a maximised amount of the non-varied class. The results for this are given in Figure 5.6 on page 47. Again the class accuracies consistently showed that AD cannot be distinguished until at least 75 samples were incorporated into the test set. Additionally, it was expected that the overall accuracies would increase towards having equal number of samples from each set. This was not the case when maximising the AD class and varying the number of NC and MCI samples. The accuracy shown here has a hump shape. After the introduction of 100 NC and AD samples, the accuracy plateaued and perhaps even showed a slight decrease. The same pattern was seen in the loss- the loss dramatically decreased with the introduction of more NC and MCI samples up to a point, but no further decrease was observed from there. When looking at the class accuracies, it was consistently demonstrated that

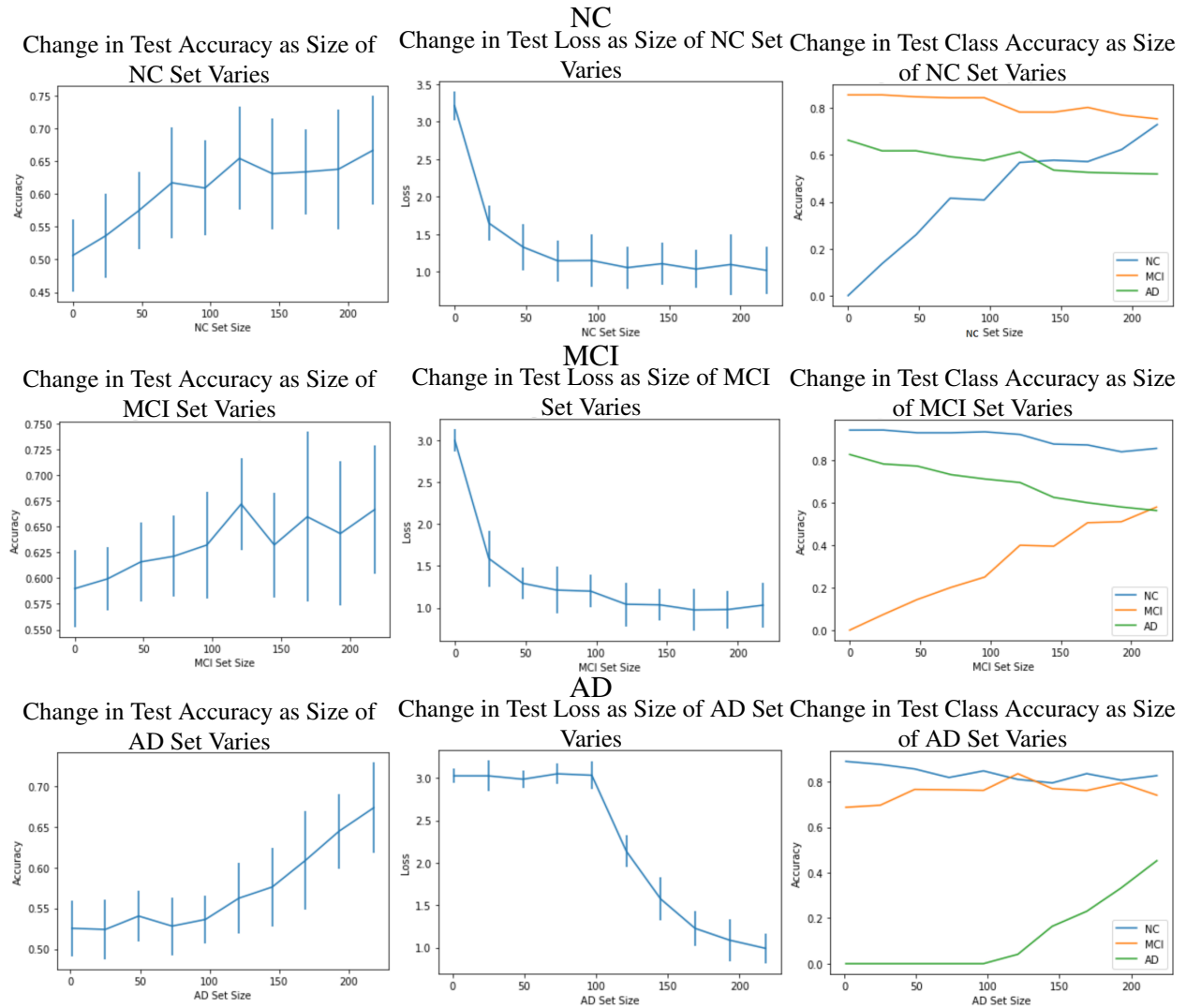


Figure 5.5: Class Imbalance Experiment - Minimising a class

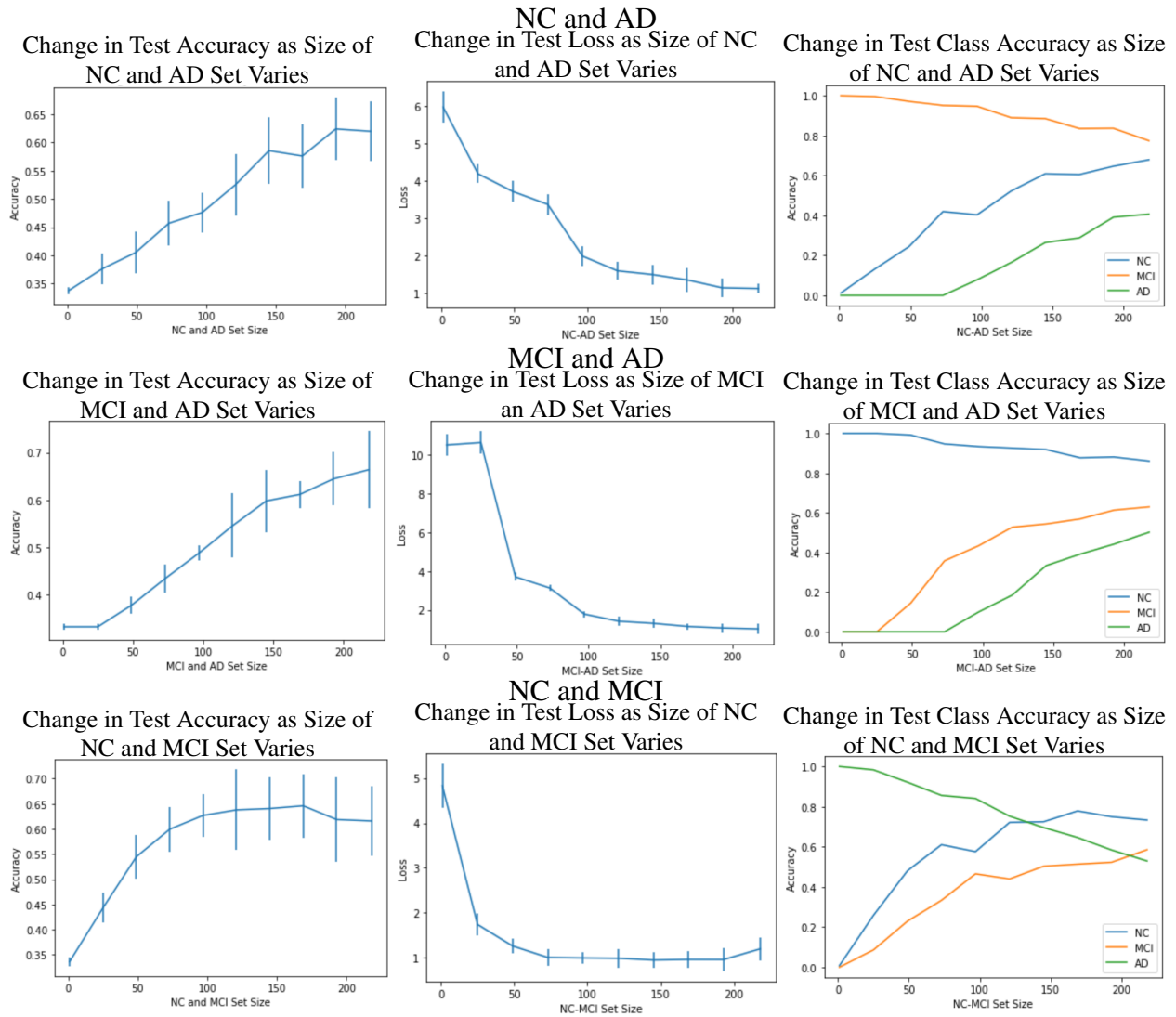


Figure 5.6: Class Imbalance Experiment - Maximising a class

NC was more easily classified than MCI or AD as the class accuracy whenever NC was one of the varied sets sat above that of the other varied set. Additionally, the AD set showed the biggest decline upon the addition of more of the other samples, whereas the decrease in accuracy of the AD and MCI class accuracies when they are maximised was less severe.

## 5.5 Data Augmentation

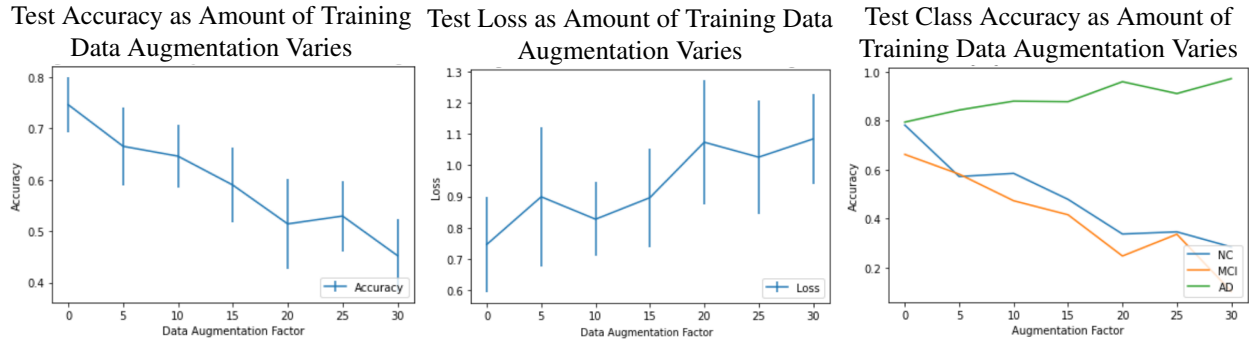


Figure 5.7: Varying the Amount of Training Data Augmentation

The amount of data augmentation was varied using the Keras Image data generator preprocessing library. This was conducted after the test and validation sets had been extracted. The augmentation performed included rotation, brightness adjustment and zoom. These factors were changed uniformly.

The results are given in Figure 5.7 and demonstrate no advantage from introducing any amount of data augmentation, except on the class accuracy of the AD class. The experiment was re-conducted on a deeper network topology to see whether a deeper network could tackle the problem better. This was not the case. The performance in training, including the training accuracy and validation also demonstrated that introducing data augmentation made the problem more difficult. This suggests that the problem is easier to solve with just the linearly registered images, and adding variation to the images does not help in solving the problem. This was a potentially concerning revelation, as it suggested that the model was struggling to learn the patterns that demonstrate the observed classes, as these patterns cannot be observed as well when the data is augmented. The augmentation experiment was performed again on the final model, adjusting just the brightness and these results are given subsequently.

## 5.6 Data Augmentation - Brightness Only on Final Model

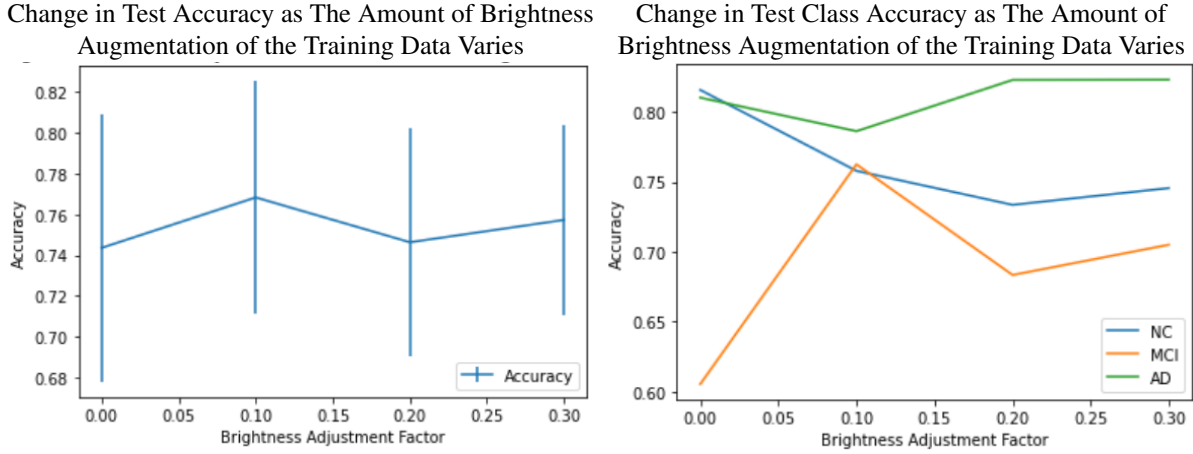


Figure 5.8: Training Data Augmentation - Brightness

This experiment on data augmentation was re-conducted after the creation of the final model, on that architecture. After observing the effects of the first experiment and taking into consideration the preprocessing already performed on the data, it was concluded that the most benefit could be observed from varying just the brightness in the images. The linear rectification of the images means that already the images are aligned and at a uniform zoom. Hence, adjusting the factors of zoom, rotation and flip in the images appears counter-intuitive to helping the learning. So when returning to this experiment using the new ensemble model, only the brightness factor was augmented in the images.

The results, given in Figure 5.8: Training Data Augmentation-Brightness, demonstrate an advantage by adjusting the brightness within the training set by a factor of 0.1. This demonstrated the most advantage in the MCI class and also brought all class accuracies within a closer range.

## 5.7 Preprocessing Results

The results of the image resizing experiment suggested that the reduction to the image to a size of  $64 \times 64$  did not dramatically reduce the quality of classification that could be made but allowed for the creation of a more complex network due to lesser constraints on both memory and time to train.

The results of the slice point experiment concluded to centre the plane extraction at the following positions:  $x = 72$ ,  $y = 118$ ,  $z = 63$

The experiment into the incorporation of multiple slices in each plane concluded to use just the single slices to perform the classification. This aligns with Payan and Montana [2015], who compared 2D and 3D processing methods and found that 2D methods could match the performance of the 3D model.

The class imbalance experiment revealed some interesting properties of the classification. The AD classification was hard to make when given too many MCI and NC samples. This reduced the overall accuracy of the model. NC appeared to be the easiest to classify. These results suggest the model is trained to easily distinguish NC and MCI but hence struggled to learn the features of the AD brain. It appears, then, that the distinction between MCI and AD was the hardest. If the final model architecture also struggled with this distinction, then a manipulation of the class imbalance was determined to ensure the features of an AD brain can be learnt. This was not required.

The data augmentation experiment first appeared inconclusive. But a follow up on the final model revealed that a small amount of brightness augmentation in the training set was beneficial, especially to the MCI classification.

# Chapter 6

## Model Construction

This section presents the experiments conducted that contributed to the structure, architecture and topology of the final model and training. This began with the establishment of the model hyperparameters and is followed by the training parameter experiments.

### 6.1 Model Hyperparameters

#### 6.1.1 Ensemble

The development of the ensemble model arose from the idea that separating the planes and keeping the images separate could yield better results than combining them into a single image. This created three separate models that could each be optimised to perform the classification on a particular physical plane.

#### Individual Plane Performance

The first step was to observe the results of the classification performed upon each plane. These are given to 3s.f. in table 6.1.

| Colour/Plane      | Test Accuracy | Test Loss | NC Accuracy | MCI Accuracy | AD Accuracy |
|-------------------|---------------|-----------|-------------|--------------|-------------|
| Red (Sagittal)    | 0.676         | 0.829     | 0.857       | 0.683        | 0.485       |
| Green (Coronal)   | 0.711         | 0.880     | 0.852       | 0.724        | 0.556       |
| Blue (Transverse) | 0.682         | 1.06      | 0.818       | 0.737        | 0.490       |

Table 6.1: A Summary of the performance of each model trained separately by plane

The coronal plane demonstrated the best overall accuracy, but all were closely bound. The

coronal plane also demonstrated best accuracy in AD, the sagittal plane demonstrated best accuracy in NC and the transverse plane was best for classifying MCI. This demonstrated that all three planes contribute different things to the classification and should all be incorporated.

### Combining Planes

The next experiment, creating an ensemble model by combining the planes, aimed to discover whether combining each of these models creates a model that is better than each plane individually and also rivals the performance of the combined image model.

| Model             | Test Accuracy | NC Accuracy | MCI Accuracy | AD Accuracy |
|-------------------|---------------|-------------|--------------|-------------|
| Combined          | 0.701         | 0.832       | 0.770        | 0.502       |
| Red (Sagittal)    | 0.621         | 0.786       | 0.712        | 0.366       |
| Green (Coronal)   | 0.686         | 0.812       | 0.73284314   | 0.516       |
| Blue (Transverse) | 0.685         | 0.814       | 0.703        | .535        |
| Ensemble          | 0.708         | 0.844       | 0.774        | 0.507       |

Table 6.2: A Summary of the performance of each model

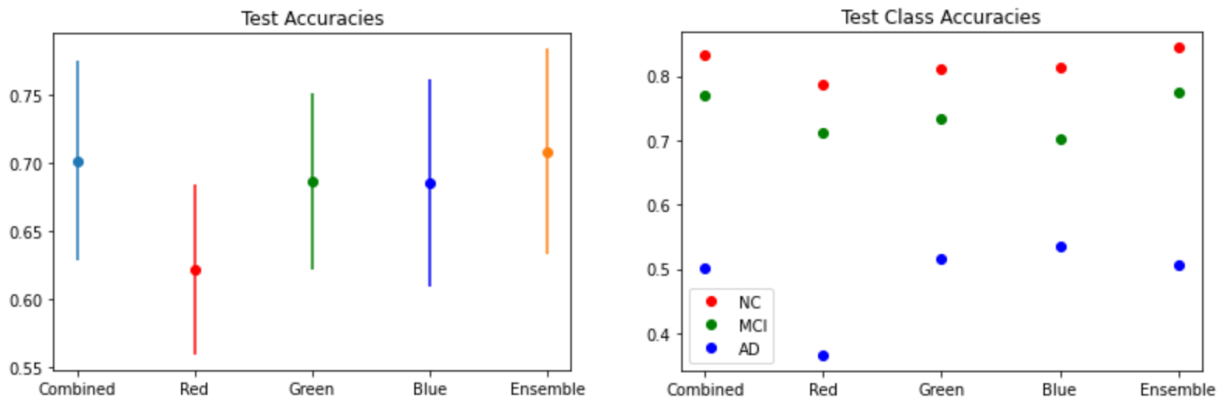


Figure 6.1: Ensemble Model Comparison

The results are given in Table 6.2 and in Figure 6.1: Ensemble Model Comparison. These accuracies were generated using exactly the same data to train each model. Although the results are subtle, they still demonstrate an improvement from separating the images and training three separate models. Additionally, this created the opportunity to optimise the structure of each model individually which will increase the advantages further.

## Ensembling Method

There are two main methods for ensembling different ML models together. The first is an averaging method and the second is weighted averaging. The difference is that in the first all models have an equal influence over the final result and in the weighted averaging, the models have different weighting over the final result depending on the accuracy they have demonstrated during training. The code demonstrating the difference between the two functions is given in appendix E. A third weighted averaging method, where the weights were chosen by hand was also observed. The results are given below:

```
Estimated Averaging Accuracy 0.771 (0.042)
Estimated Weighted Averaging Accuracy 0.769 (0.043)
Self Determined Weighted Averaging Accuracy 0.748 (0.060)
-----
Red Accuracy 0.709 (0.045)
Green Accuracy 0.742 (0.061)
Blue Accuracy 0.752 (0.052)
```

These show that using non-weighted averaging produced a better result. This further means that even if a model demonstrated bad accuracy overall, it can still have a good class accuracy in one class and provide valuable insight within this.

### 6.1.2 Random Search Model Optimisation

For each of the physical planes, a random search was conducted over 80 trials. The code that was used in generating the different models is given in appendix D and a summary of the hyperparameters and their values is given in Table 6.3. The batch size and number of epochs were also varied. Parameters such as filter size, kernel size, activation function and dropout rate varied independently within each layer where they were present.

| Hyperparameter         | Values                    | Default   |
|------------------------|---------------------------|-----------|
| Filters                | 16 to 128 in steps of 32  | 16        |
| Kernel Size            | 3,5,7                     | 3         |
| Activation Function    | Relu, Tanh, Sigmoid       | Relu      |
| Conv-Pool Layers       | 1, 2, 3                   | Null      |
| Pool Size              | 1,2                       | 2         |
| Dropout rate           | 0 to 0.4 in steps of 0.05 | 0.25      |
| Number of Dense Layers | 1, 2, 3                   | Null      |
| Dense Units            | 32 to 512 in steps of 32  | 128       |
| Learning Rate          | $10^{-4}$ to $10^{-2}$    | $10^{-3}$ |
| Epochs                 | 15 to 50 in steps of 5    | Null      |
| Batch Size             | 32 to 256 in steps of 32  | Null      |

Table 6.3: The Hyperparameters Changed in Random Search

The number of models to be compared could be reduced to three architectures that resulted from these searches. The top three results for each of the optimisations run (red/green/blue) convolved i.e. the top result for the green (coronal) plane was also in the top three for the red (sagittal) plane, and the same for the blue (transverse) plane. These architectures are given in appendix F. They are named ‘A’, ‘B’ and ‘C’ with ‘A’ being the result of random search on red, ‘B’ being the result of random search using green and ‘C’ being the result of random search on blue.

The models generated by random search were also compared to the model used by Dai [2019] and the basic model used for training, given in appendix C. The results are given in Figure 6.2: Test Accuracies of All Network Topologies, given below.

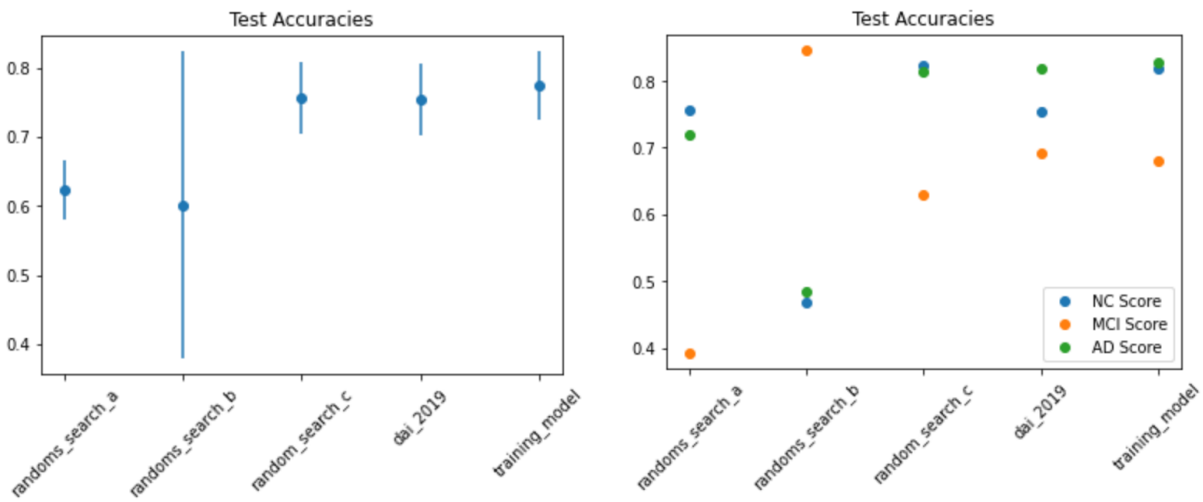


Figure 6.2: Test Accuracies of All Network Topologies

Surprisingly, the basic model intended only for efficiency when running experiments demonstrated the best performance. Model ‘B’ from the green plane random search demonstrated a particular affinity for the MCI class, however demonstrated huge variability.

After discounting models, ‘A’ and ‘B’, two things remained: evaluate the performance on each plane, and visualise the training.

The validation accuracies from each model on each plane are given Table 6.4.

| Plane \ Model | Randoms Search Result C | Dai 2019      | Training Model |
|---------------|-------------------------|---------------|----------------|
| Red           | 0.678(0.0263)           | 0.642(0.0220) | 0.722(0.0220)  |
| Green         | 0.773(0.0200)           | 0.779(0.0234) | 0.767(0.0299)  |
| Blue          | 0.742(0.0217)           | 0.751(0.0367) | 0.753(0.0252)  |

Table 6.4: Accuracy Performance for Each Plane from each model

The training model demonstrated the best performance on the red and blue planes, however the Dai [2019] model demonstrated best performance where the training model performance lacked, in the green (coronal) plane. This suggests a deeper, more complex, model is required for the classification performed on this plane. A model was created that combined these two models, using the training model for the red and blue planes but the Dai [2019] model for the green plane. This demonstrated no advantage compared to that of the training model on its own. Hence, the training model was used for further investigation and improvement.

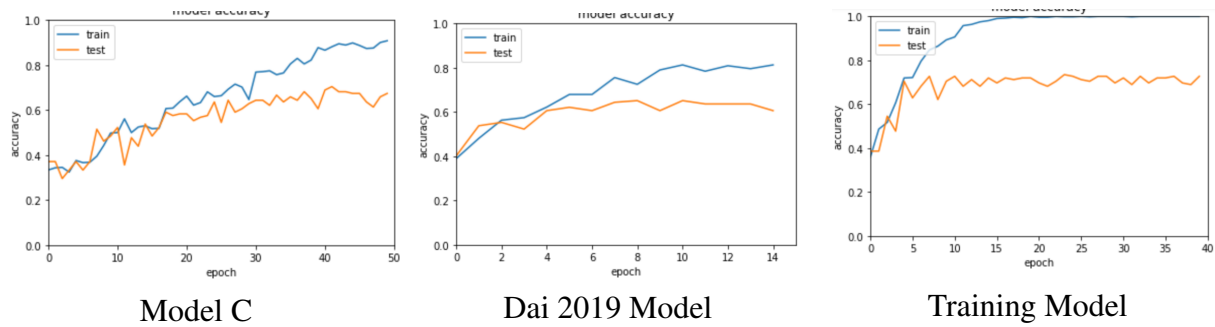


Figure 6.3: Training and Validation Accuracy of Models over Training

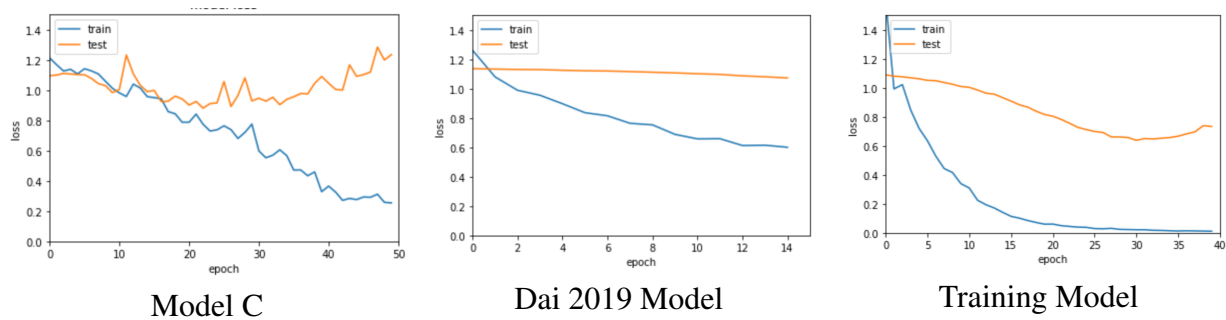


Figure 6.4: Training and Validation Loss of Models over Training

From the training graphs (Figure 6.3 and Figure 6.4 on the next page), it was clear that all models demonstrated overfitting. The large gap between the validation and training metrics also demonstrated a under representative training set. The training model demonstrated the most promising performance in the training data and showed good stability. Whereas the 'C' and Dai [2019] graphs show a training loss that continues to decrease and does not settle. A good model will have closely related training and validation metrics that settle to a point of stability. The training model is the only one that comes close to showing this, and, additionally, it is the simplest model, which should be a step towards solving the overfitting problem. By Occam's razor, keep the simplest model that gives the required level of performance.

### 6.1.3 Batch Normalisation

A batch normalisation layer aims to reduce the generalisation (testing) error and also improve the stability of training by standardising the inputs to a layer. There were a number of factors to consider within this section.

- Number of batch normalisation layers

This experiment investigated the effect of incorporating batch normalisation layers and compared the following architectures: No batch normalisation layer, a single batch normalisation before flattening, and finally two batch normalisation layers.

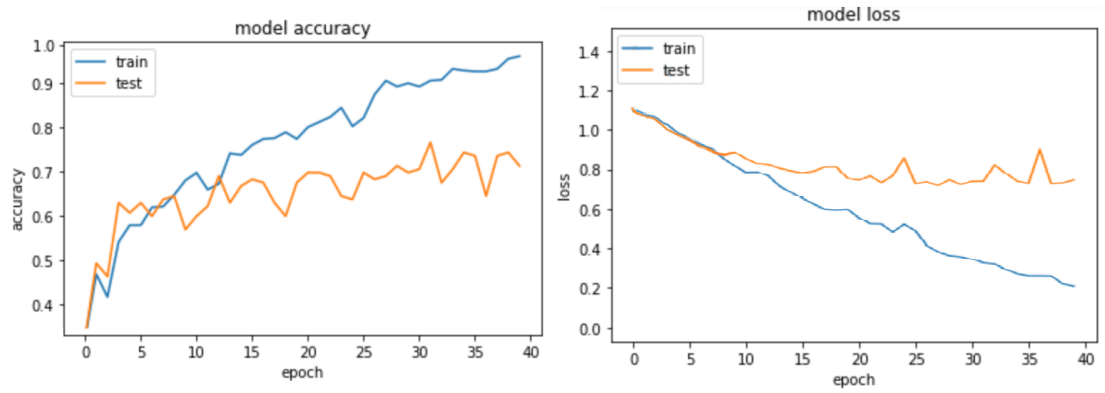


Figure 6.5: Training of Model with No Batch Normalisation

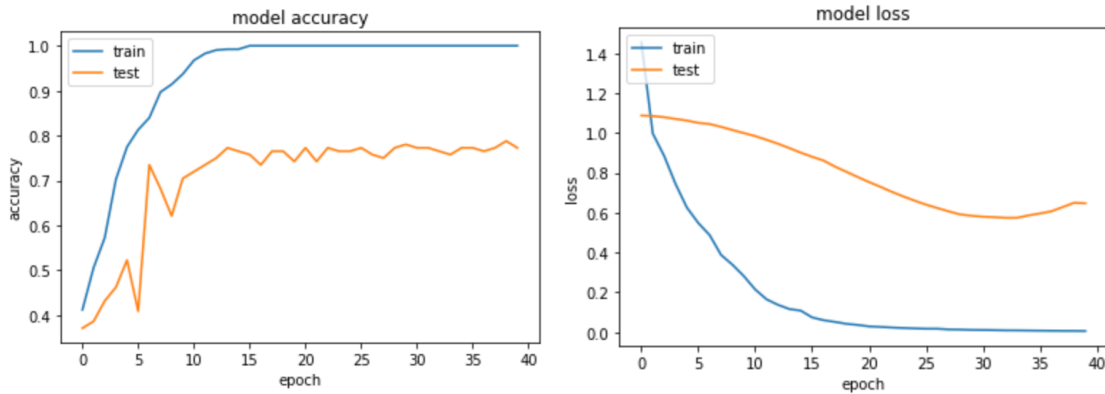


Figure 6.6: Training of Model with Single Batch Normalisation Layer

The resulting training graphs (for the two successful models) are given above in Figure 6.5 and Figure 6.6. The accuracies for the models with left being no batch normalisation layers, centre being a single batch normalisation and right being two batch normalisation layers were:

Accuracies=[0.724,0.743,0.372] (3s.f.)

A single batch normalisation layer provided the desired outcome. There was less variability over training and the resulting test accuracy was better. Adding a single batch normalisation layer also made the class accuracies more consistent across all three classes. More than one batch normalisation layer meant the model could not learn.

- Placement

The placement of the batch normalisation layer was varied. It was placed before the activation function, after the activation function and after the pooling layer.

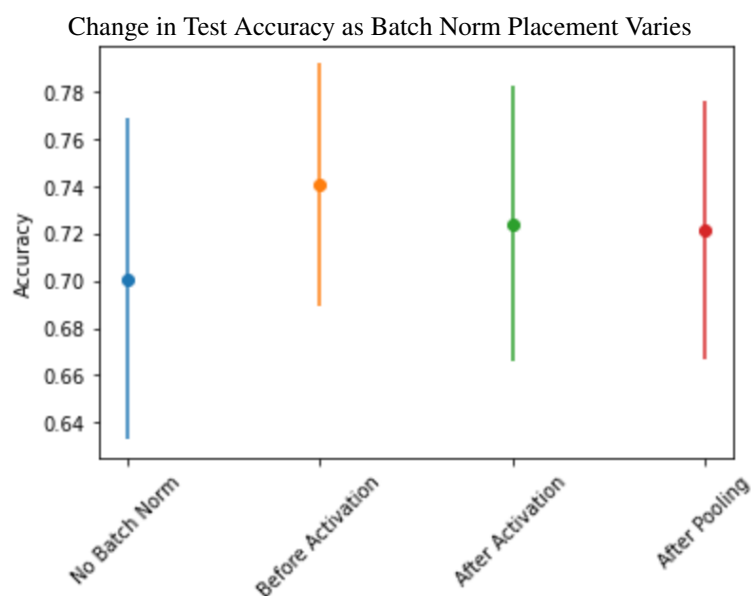


Figure 6.7: Test Accuracy when Varying Batch Norm Placement

The results, given in Figure 6.7 show the best performance was seen with the single batch normalisation layer placed before the activation function

- Without Dropout

As batch normalisation is a method of regularisation it was also implemented into the model without any of the dropout layers, as a batch normalisation layer should, in theory, eliminate the need for a dropout layer.

The result was that with dropout remaining in the model, placing the batch normalisation after the activation function came out best. However, with dropout removed, placing the batch normalisation before the activation function demonstrated the best performance. As it is likely that some form of dropout will be added following future investigations, the activation function remained after the activation function.

Final Batch Normalisation Outcome: A Single Batch Normalisation function, placed after the activation function

### 6.1.4 Dropout

To observe the effects of dropout, a grid search was run with dropout amounts of 0,0.25 and 0.5. The dropout layers were placed after each of the two pooling layers and the dense layer, resulting in three dropout variables.

The Top Five Results are given in the table below:

| Dropout Amounts             | Test Accuracy (std) | NC Acc. | MCI Acc. | AD Acc. |
|-----------------------------|---------------------|---------|----------|---------|
| one=0, two=0, three=0.25    | 0.760 (0.0652)      | 0.806   | 0.696    | 0.777   |
| one=0, two=0.25, three=0    | 0.759 (0.0537)      | 0.761   | 0.695    | 0.819   |
| one=0, two=0.5, three=0.25  | 0.759 (0.0432)      | 0.761   | 0.674    | 0.839   |
| one=0, two=0, three=0.5     | 0.755 (0.0587)      | 0.782   | 0.691    | 0.789   |
| one=0, two=0.25, three=0.25 | 0.754 (0.0425)      | 0.770   | 0.687    | 0.805   |

Table 6.5: Accuracy Performance when Varying Dropout Amounts

This demonstrated that some dropout does improve performance. It worked best if implemented a) in small amounts, and b) in the later layers of the architecture. A single dropout layer was used after the dense layer of the final model and the single batch normalisation resulting from the previous experiment was used following the convolution and pooling layers.

### 6.1.5 Network Depth

When viewing the affects of depth the number of parameters (i.e. network weights) were kept the same, but the number of layers and the distribution of the parameters (through filter size) were modified. Making the model deeper but having the same number of parameters has been shown to improve a models ability to generalise [Wu et al., 2017], which aimed to help in the challenge of bringing together the validation and training metrics to reduce overfitting.

#### Conv-Pool Layers



Figure 6.8: Changing Depth in the Conv-Pool layers Results

When changing the depth in the conv-pool cycles, four models were created:

- A) The Original Model. Four convolutional layers paired in twos followed by pooling layers with filters (16,16,32,64)

- B) Single convolution layers. Two convolutional layers after the input each followed by pooling with filters (16,16,55)
- C) Flat Model. Just the input layer and one other convolutional layer with filters (12,12)
- D) More Layers. Attempted to create a model with more paired layers but couldn't retain the number of parameters. Changed the number of filters, retainin the number of parameters instead. The resulting model had conv filters (32,64,256,256)

The results, as given in Figure 6.8 at the top of this section, demonstrate that the best model, with the highest overall accuracy (and also across all classes) was model D, with only a single convolution layer in the second cycle but an increased number of filters to make up the parameters. The training performance for this model has also been included as it demonstrates a much better ability to reduce the validation loss during training compared to the others. The resulting model is given in appendix G.

### Dense Layers

Again, the depth of the dense layers was modified such that the number of model parameters didn't change by modifying the number of neurons in each layer. The model used for this experiment was model D from the experiment above. The number of neurons in each layer after the flattening layer for each model is given below:

- Model A (Original): 448,3
- Model B: 224, 224, 3
- Model C: 410, 410, 410, 3
- Model D: 448, 512, 32, 3
- Model E: 32, 32, 3

The results are given in Figure 6.9: Changing Depth, and Table 6.6. Despite more variability, model C was chosen because of

| Model Name | Test Accuracy | Deviation | NC Accuracy | MCI Accuracy | AD Accuracy |
|------------|---------------|-----------|-------------|--------------|-------------|
| C          | 0.786         | 0.0556    | 0.803       | 0.741        | 0.815       |
| D          | 0.782         | 0.0363    | 0.832       | 0.664        | 0.852       |
| B          | 0.770         | 0.0545    | 0.786       | 0.704        | 0.819       |
| A          | 0.765         | 0.0561    | 0.807       | 0.668        | 0.824       |
| E          | 0.764         | 0.0442    | 0.827       | 0.639        | 0.828       |

Table 6.6: A Summary of the performance of each model with varying dense layer depth (3s.f.)

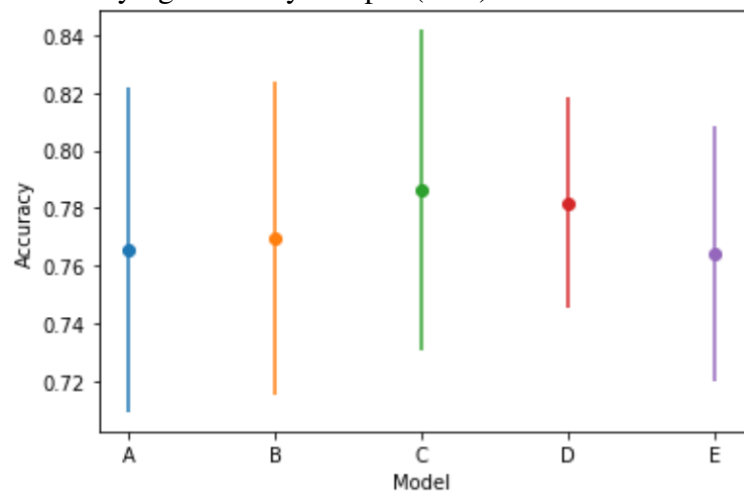


Figure 6.9: Changing Depth in the Dense layers Results

the better performance between each of the classes. Model D had less variability but showed much worse performance at classifying the MCI samples. The resulting model is given in appendix H.

### 6.1.6 Filters

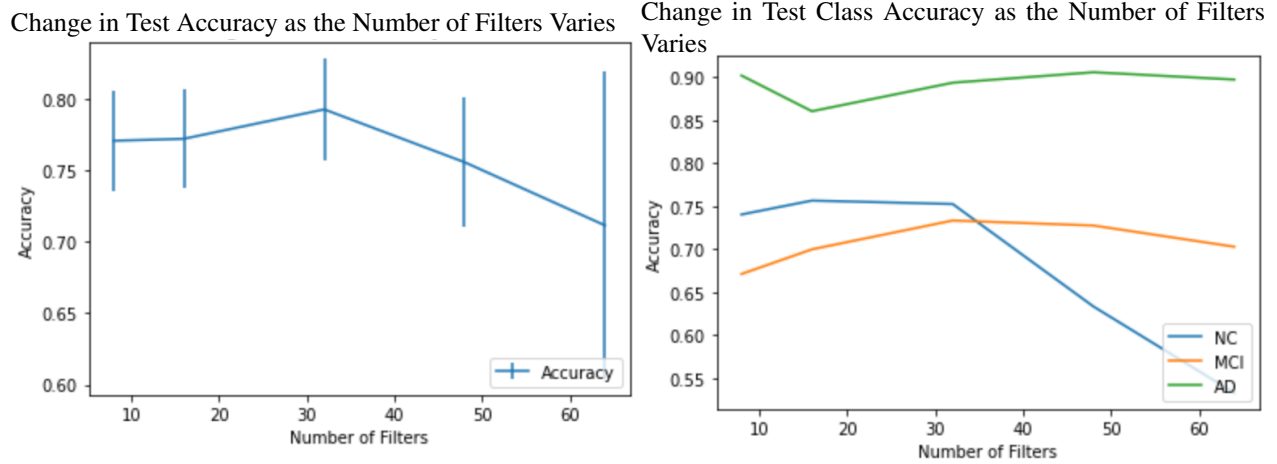


Figure 6.10: Varying the Number of Filters

Varying the number of filters affects the number of parameters. The shape of the network was retained by creating a model based on a single filter parameter but scaling it up through the structure in lieu with the model in appendix H. So taking  $x$  as the number of filters in the input layer, the second convolutional layer had  $x \times 2$ , the third  $x \times 8$  and the fourth  $x \times 8$  also. Because changing the number of filters changes the number of parameters that the model must change, the patience of the early stopping was reduced to allow a quicker halt to training if the model required fewer epochs. The models, their filter parameter and the number of model parameters are given in the table below:

| Model Name | Filters (First layer) | No. Model Parameters |
|------------|-----------------------|----------------------|
| A          | 8                     | 1,331,159            |
| B          | 16                    | 2,418,119            |
| C          | 32                    | 4,875,431            |
| D          | 48                    | 7,718,599            |
| E          | 64                    | 10,922,599           |

Table 6.7: Models and their Number of Parameters for Filter Amount Experiment

Figure 6.10 at the start of the section gives the results. These demonstrate that the original model structure had the best number of filters for the problem. It was interesting to see that the NC

class accuracy declines dramatically as a larger number of filters are introduced.

### 6.1.7 Kernel Size

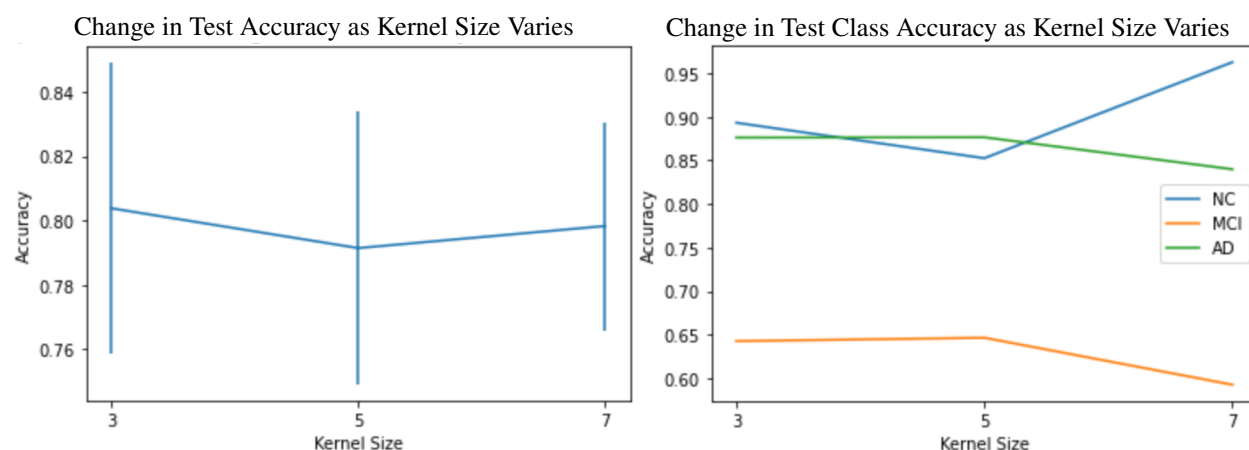


Figure 6.11: Changing Kernel Size Results

Before changing the kernel size for every layer individually, all convolutional kernels were changed uniformly to see what the effect would be. The results, as given in Figure 6.11: Changing Kernel Size Results, demonstrate that there was no considerable effect from changing the kernel size. It appears that increasing the kernel size reduced the class accuracies for MCI and AD while increasing the performance for NC.

### 6.1.8 Padding

Two models were created, one with padding and one without. The model used previously had no padding. The results were as follows:

```
----- End of K fold Results -----
Model with No Padding Test Accuracy 0.801 (0.079)
Model with No Padding Test Class Accuracy [0.889      0.692      0.82316667]
-----
Model With Padding Test Accuracy 0.806 (0.059)
Model with Padding Test Class Accuracy [0.87216667 0.74866667 0.798      ]
-----
```

Although the accuracies were extremely close, the smaller variability and the better MCI class accuracy resulted in the decision to add padding in the final model.

### 6.1.9 Pooling

| Model Name | Test Accuracy | Deviation | NC Accuracy | MCI Accuracy | AD Accuracy |
|------------|---------------|-----------|-------------|--------------|-------------|
| 2,2,2      | 0.804         | 0.0456    | 0.892       | 0.712        | 0.806       |
| 3,3,2      | 0.804         | 0.0583    | 0.814       | 0.742        | 0.856       |
| 3,2,2      | 0.798         | 0.0359    | 0.872       | 0.692        | 0.831       |
| 2,2,3      | 0.794         | 0.0540    | 0.789       | 0.732        | 0.860       |
| 3,2,3      | 0.768         | 0.0489    | 0.729       | 0.711        | 0.864       |
| 2,3,3      | 0.767         | 0.0470    | 0.711       | 0.736        | 0.851       |
| 3,3,3      | 0.756         | 0.0750    | 0.687       | 0.671        | 0.909       |

Table 6.8: A Summary of the performance of each model with varying pooling sizes (3s.f.)

When varying the amount of pooling, out of memory exceptions were encountered. So the Experiment could only be run varying the pool size of 2 or 3 after each convolutional layer. This created 3 pooling layers, one after each convolution layer (a pooling layer was not used after the input layer).

The results, shown in Table 6.8 demonstrate little variation, except when a higher pool size (3) is used in the later layer. This suggests that pooling is more beneficial within the early layers of the network. At a pool size of 3 in all layers, the class accuracies for both MCI and NC tumble, while the class accuracy for AD soars, suggesting some ‘guessing’ from the model towards the AD category.

This was further investigated by removing the pooling layer from the last convolution. The only model matching the performance was that with the first two pool sizes as 3, which achieved an accuracy of 0.809 (0.0554) to 3s.f. and class accuracies of [0.921, 0.701, 0.806] (again, 3s.f.). This was investigated further by moving the batch normalisation to before the activation function, as had worked better without the pooling in the batch normalisation experiment. This model achieved an accuracy of 0.790, (0.0495) and class accuracies of [0.802, 0.711, 0.856] (3s.f.).

It remained that models with (pool size 1, pool size 2, pool size 3) as (2,2,2), (3,3,2) and the first (3,3) models had comparative performance. The (3,3,2) model has a considerably better MCI class accuracy and produces a model with the fewest parameters. Therefore, because of the highlight in the significance of the MCI classification pool sizes of 3, 3, and 2 were chosen. The resulting model had significantly fewer parameters, at 2,040,935 compared to the 4,873,235 as it had before.

### 6.1.10 Activation Functions

Two experiments were run here, first changing the activation function of the convolutional layers, then second changing the activation function of the dense layers. Before the experiment, the convolutional layers were followed by ReLu activation and the dense layers followed by tanh activation.

The results given in Figure 6.12 on the next page demonstrate the best model was achieved using ReLu activation functions on both convolution and dense layers. This produced the model with the best accuracy and with the closest matching, as well as highest class accuracies.

The experiment was then furthered to see if there was an advantage from using Leaky ReLU activation as in Dubey and Jain [2019] and Zhang et al. [2017]. This demonstrated further improvement in the accuracies given below:

|                                      |             |            |             |
|--------------------------------------|-------------|------------|-------------|
| Estimated Relu Model Accuracy        | 0.775       | (0.061)    |             |
| Estimated Relu Model Class Accuracy  | [0.81366667 | 0.69166667 | 0.8195 ]    |
| -----                                |             |            |             |
| Estimated Leaky Relu Model Accuracy  | 0.785       | (0.064)    |             |
| Estimated Leaky Model Class Accuracy | [0.84333333 | 0.69866667 | 0.81166667] |

Hence, all activation functions in the model will be LeakyReLU.

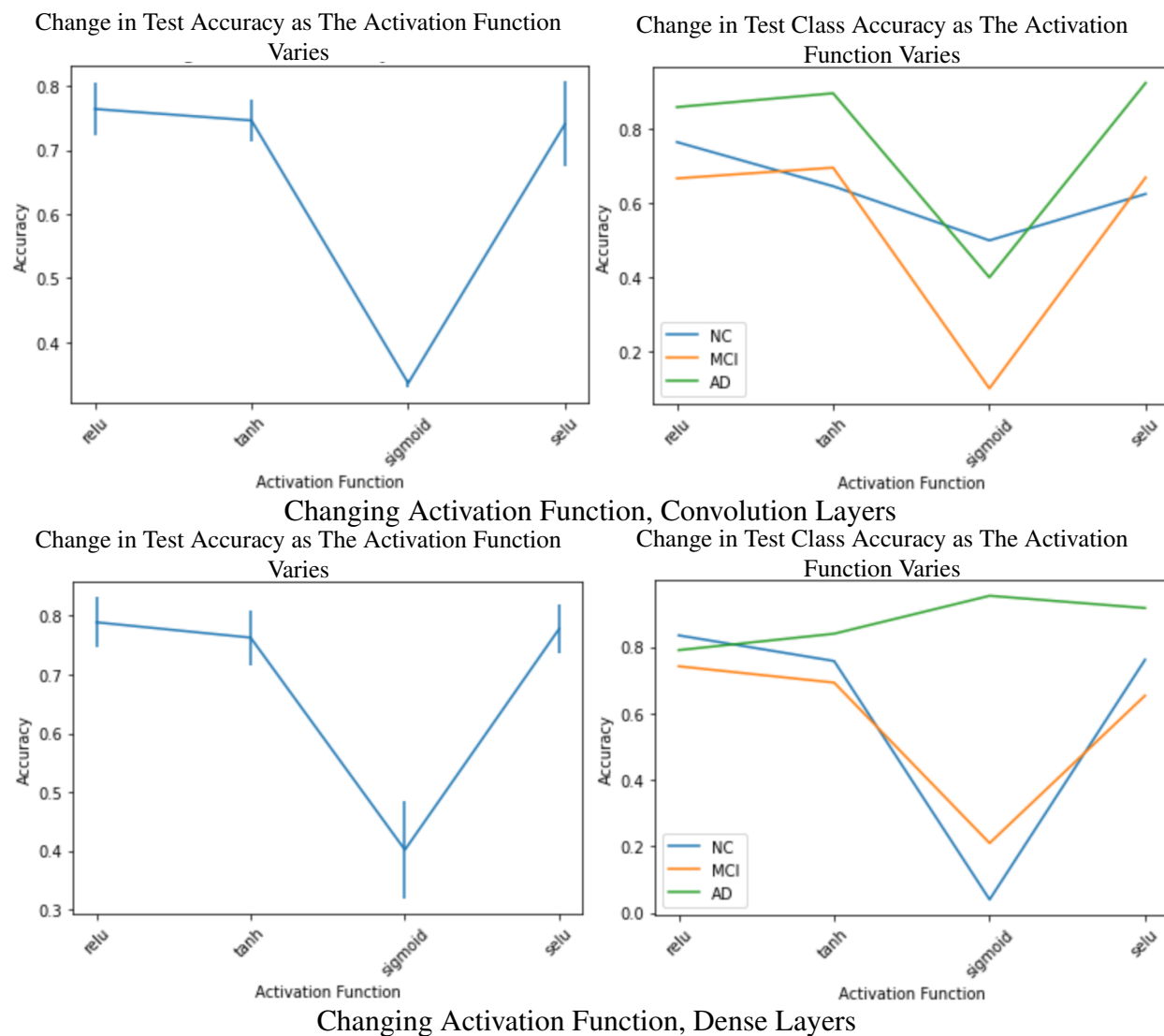


Figure 6.12: Changing the Activation Functions

## 6.2 Training Hyperparameters

### 6.2.1 Early Stopping

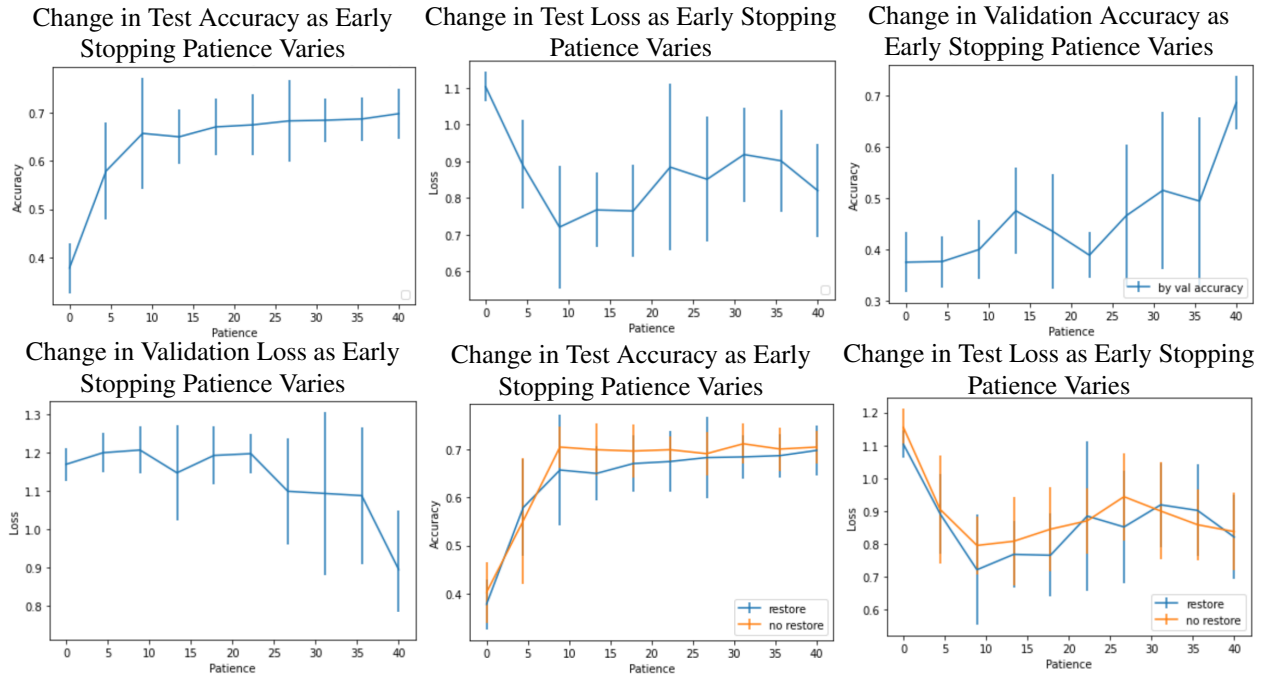


Figure 6.13: Early Stopping - Varying Patience

The early stopping experiments were conducted to answer a number of questions: should validation loss or validation accuracy be chosen as a metric, what should be patience level be set to, and should weights be restored to their values before the decline in performance.

The results, in Figure 6.13:Early Stopping, demonstrate that validation loss is a more reliable metric. Stopping the training early using a lower patience using validation accuracy as the metric always resulted in a worse model than had early stopping not been implemented (patience = epochs = 40). So validation loss was chosen as the metric.

Although the average loss was optimum at a patience of 10 and the accuracy was also promising here, the error bar shows there was too much variance. A slightly larger patience value of 15 was more reliable.

Following from this, the option of restoring network weights was considered. Not to restore was the chosen outcome because the accuracy was better on every account, whereas loss was more variable, not restoring the network weights was the selected method.

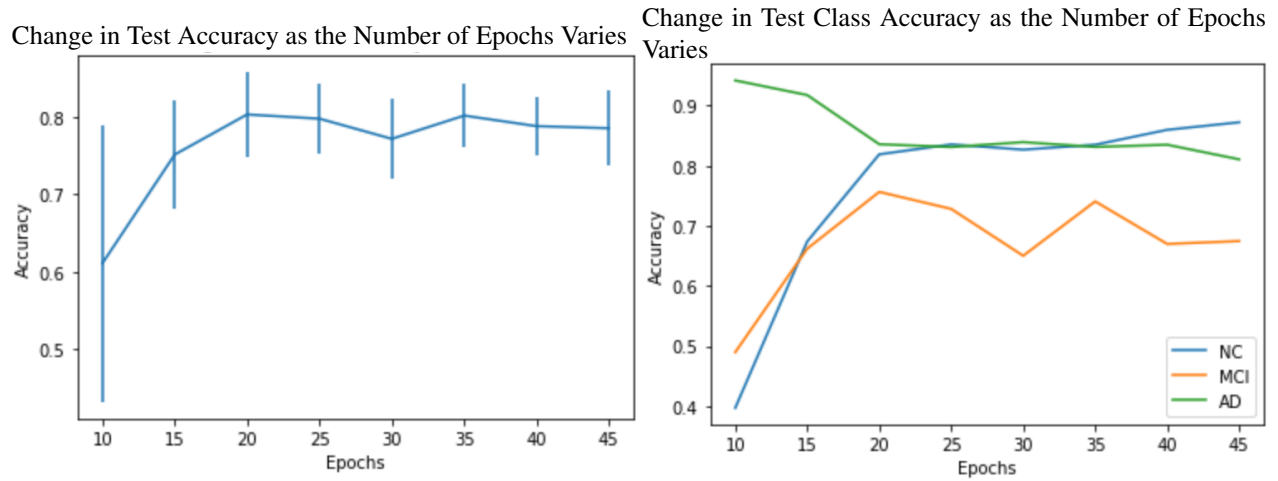


Figure 6.14: Varying the Number of Epochs

### 6.2.2 Epochs

For this experiment, the early stopping was removed. Given the new model structure, it was important to re-evaluate the number of epochs over which training is performed.

The results, given in Figure 6.14: Varying the Number of Epochs, demonstrate that the best accuracy was achieved at 20 epochs and an increase in this number did not affect the overall accuracy but did decrease the accuracy in the MCI class.

### 6.2.3 Learning Rate and Batch Size

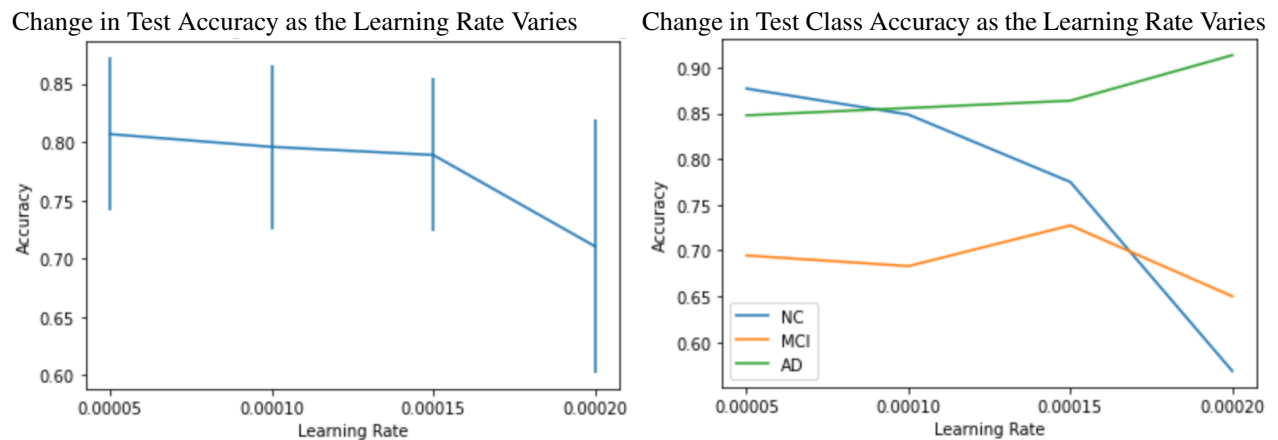


Figure 6.15: Varying the Learning Rate

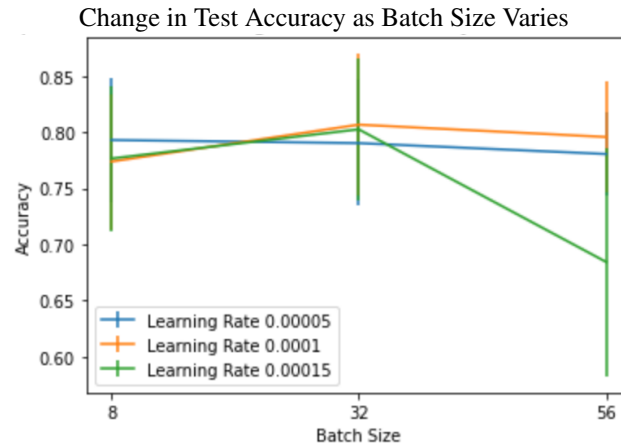


Figure 6.16: Changing Batch Size Results

A possible way of reducing the generalisation (validation) error is to reduce the batch size. A smaller batch size finds better minimums in the sense of validation [Keskar et al., 2016]. The learning rate goes hand in hand with the batch size as the batch size determines how many samples to update after and the learning rate determines how much to update by.

At first, just the learning rate was varied and the results are given in Figure 6.15: Changing Batch Size Results, above. It was clear that the problem suited a lower learning rate.

The results when changing the batch size are given in Figure 6.16 and demonstrated that the problem also suited a lower batch size, and a batch size of 32 was small enough.

Hence a learning rate of 0.00005 and batch size of 32 were adopted.

## 6.3 Model Construction Results

The final model was an ensemble of three models, each one trained on a separate physical plane. These models were ensembled using averaging. There was a single batch normalisation layer after the last convolution and before the last pooling layer. There were three conv-pool cycles after the input layer. A dropout of 0.25 was added before the output layer and following from three densely connected layers. The filters, kernel sizes and pool sizes are given below, and each convolution layer uses padding. Every layer used a Leaky ReLU activation function.

Each model trained for 20 epochs using a batch size of 32 with early stopping implemented with a patience of 5 (since the number of epochs was reduced) and a learning rate of 0.00005.

The model structure is as follows:

Input Convolution layer: 32 filters,  $3 \times 3$  Kernel, leaky ReLU activation, padded

Convolution layer: 64 filters,  $3 \times 3$  Kernel, leaky ReLU activation, padded

Pooling Layer:  $3 \times 3$  pool size

Convolution layer: 256 filters,  $3 \times 3$  Kernel, leaky ReLU activation, padded

Pooling Layer:  $3 \times 3$  pool size

Convolution layer: 256 filters,  $3 \times 3$  Kernel, leaky ReLU activation, padded

Batch Normalisation

Pooling Layer:  $2 \times 2$  pool size

Flatten

Dense layer: 410 Neurons, leaky ReLU activation

Dense layer: 410 Neurons, leaky ReLU activation

Dense layer: 410 Neurons, leaky ReLU activation

Dropout: 0.25 Drop Amount

Output Layer: 3 Neurons, Softmax activation

The code and model summary are given in appendix I

# Chapter 7

## Results and Discussion

### 7.1 Results

For the final evaluation, the number of splits of the k fold validation was set to the highest possible number. This was 243, as stratified k-fold validation was used to ensure the number in each class was the same. This meant that for each loop, there were 726 training samples and three test samples (one from each class). Of this, 145 were used for validation and 580 for testing (again, stratified to retain an equal number in each class). The selection of the 243 samples from each class is a factor that could mean different evaluations yield slightly different results depending on which 243 samples were selected. The training and test data for each fold was saved for reproducibility. Tensorboard was not used to graph the training due to the memory requirements that saving all the log files would require. Instead, the training accuracy and loss graphs were saved sporadically, approximately every 1/100 training. This resulted in a total of 243 ensemble models, each made up of three models, one for each physical plane. The results are analysed below.

#### 7.1.1 Training

The training accuracy and validation graphs for a randomly selected fold are given in Figure 7.1 on the next page.

The accuracy during training was promising: the training and validation accuracy follow a similar trajectory but the gap between the training and validation accuracies remained large.

The loss of the validation set during training did not decrease at the same rate as the training loss demonstrating that the overfitting remained. Overall it is clear that there is still improvement to be made.

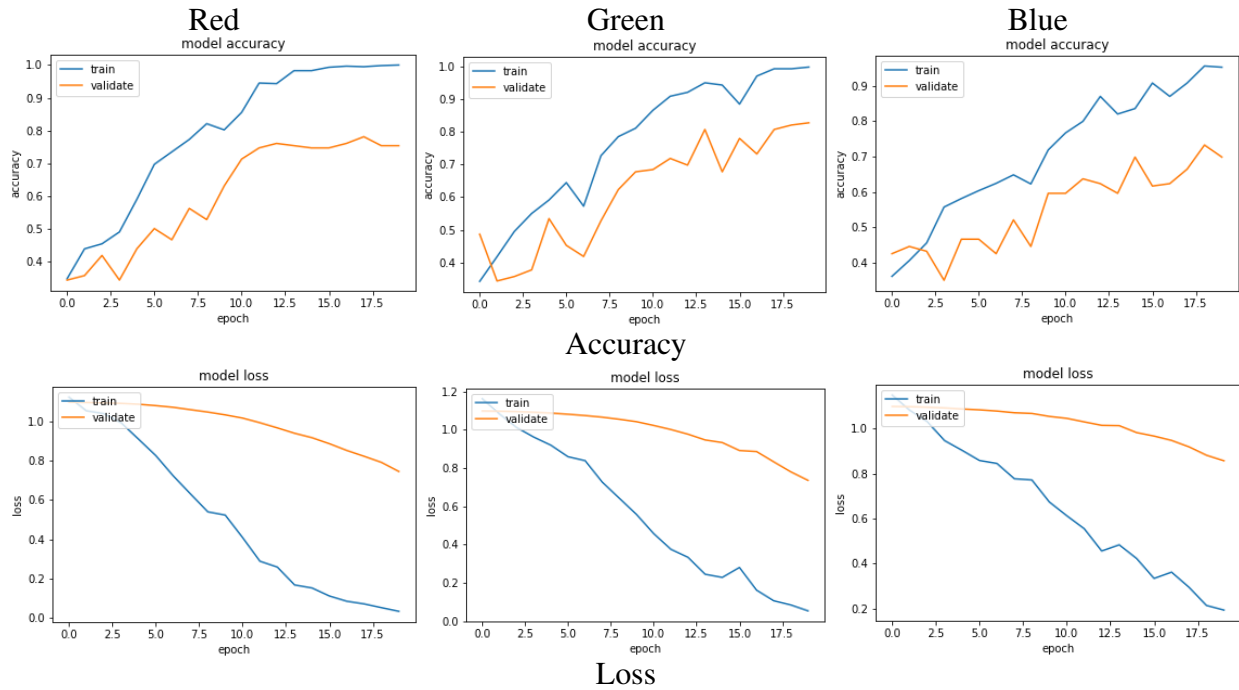


Figure 7.1: Final Model Training, Randomly Selected Fold

## 7.1.2 Metrics

Table 7.1 gives the values for a number of performance metrics achieved by the final model. The ensemble model demonstrated better accuracy performance than any of the singular plane models and with less deviation. The best accuracy was demonstrated by the green (coronal) plane.

The model demonstrated the best accuracy in the AD class, so there was no need for an increase towards class imbalance for this set. The hardest classification to make was the MCI distinction, as has been demonstrated in the literature. The coefficient of variation fell within an acceptable amount, at 0.26929883, however a lower standard deviation would have been desirable.

The scores in f1, precision and recall were generally good and the combined ROC was very promising. The log loss was higher than would be desired but still good considering the 'dumb' log loss is 1.10 in the 3 class multi classification state (meaning it chooses randomly). 0.4314 is considerably less than this, meaning the model has learnt and was making informed decisions.

| Metric    |       | Score              | Standard Deviation  |
|-----------|-------|--------------------|---------------------|
| Accuracy  |       | 0.8065843621399177 | 0.21721223408060325 |
| Accuracy  | Red   | 0.7695473251028807 | 0.23604329876049363 |
|           | Green | 0.7832647462277091 | 0.2315604701894013  |
|           | Blue  | 0.7777777777777777 | 0.2263000343188069  |
| Accuracy  | NC    | 0.8436214          | 0.3632139           |
|           | MCI   | 0.72427984         | 0.44687644          |
|           | AD    | 0.85185185         | 0.35524678          |
| F1 Score  | Micro | 0.8065843621399177 | 0.21721223408060325 |
|           | Macro | 0.7569730224051211 | 0.26011327620794367 |
| Precision | Micro | 0.8065843621399177 | 0.21721223408060325 |
|           | Macro | 0.7322816643804299 | 0.2829655691220659  |
| Recall    | Micro | 0.8065843621399177 | 0.21721223408060325 |
|           | Macro | 0.8065843621399177 | 0.21721223408060325 |
| ROC       |       | 0.9492455418381345 | 0.10979936631501663 |
| Log Loss  |       | 0.4314394735664753 | 0.36143710649911615 |

Table 7.1: Final Model Evaluation Metrics

### 7.1.3 Confusion Matrix

In the confusion matrix, given in Figure 7.2, a better visualisation and assessment of where the model was getting things wrong can be gained. From this is it clear that the most errors occurred between neighbouring classes, such as NC classified as MCI, and this explains why a lower accuracy for the MCI classification occurred- it has two neighbouring classes. More MCI samples were classified as NC than AD, demonstrating the features that cause the distinction between these classes should be further investigated. The NC/ AD distinction was easily understood by the model.

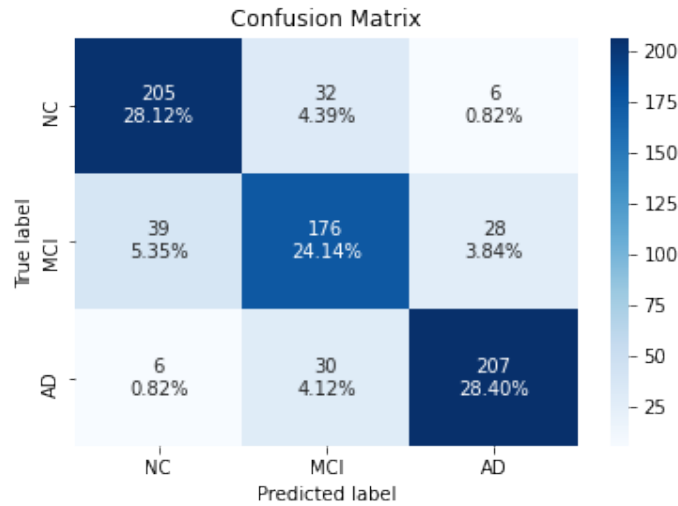


Figure 7.2: Confusion Matrix

## 7.2 Discussion

### 7.2.1 Discussion and Appraisal of Results and Methods

The results demonstrate that the created model can classify brain scans into AD, MCI and NC with an overall accuracy of 80.7% (3s.f.). The ensemble produced an improvement compared to the individual models for each plane, of which the green (coronal plane) model showed the best accuracy. This aligns with the idea that by separating the planes into three separate models, each should be able to specialise by creating unique weights within the model for the image presented from that plane. It also demonstrates that the atrophy of AD and MCI are evident in various regions of the brain, as in the work of Gupta et al. [2019] and that regional based methods for the identification of AD are effective, as in Rathore et al. [2017]. A two dimensional approach was followed, as in Payan and Montana [2015] but some three dimensional data was retained through the use of an ensemble of the planes.

The model created a three way classification into AD, MCI or NC, whereas a lot of the previous methods investigated focused upon a binary classification of just two of these sets. The effects of this were seen as MCI proved to be the hardest to distinguish from its neighbouring classes. This can be seen both in the class accuracy and the confusion matrix.

The problem suited a model architecture with small kernels and numerous layers. Generalisation methods like batch normalisation and dropout proved very helpful. Overall, it was models with a smaller number of parameters that proved successful. The final model had 2,040,935 parameters, 2,040,423 of which were trainable. It was easy to overfit and this remained a problem throughout. Hence, a smaller number of parameters, and a simpler model with lots of generalisation techniques

proved most successful.

In terms of the methods used, Keras, Tensorflow and python were a powerful team. Tensorflow provided the ability to easily run the processing on a GPU when it was available. Having a GPU was a fundamental factor in being able to improve efficiency and the work that could be completed for the project. Before it was obtained, any experiments took so long that many would have become unfeasible to run except for overnight and weekends, especially with k-fold validation to minimise noisy results. Cloud services such as google collab could also have been used and would have improved the efficiency of the project before the GPU was available.

There were elements of the project that were surprising. From general reading and research, it was expected that data augmentation would be beneficial, but after performing the experiments and thinking about its application within the context of medical imaging, it makes sense why it was not- manipulating the images breaks a heavily regulated pattern. The images were linearly registered ROI masked MRI scans. Each image has been conformed to the shape and structure of a template brain. Keeping the images in this unmanipulated state allowed any deviation from the pattern of a standard, healthy brain to be more easily detected.

In comparison with the work of previous years, a significant improvement has been achieved. Guo [2020] achieved an accuracy of 76.3% (3s.f.), Bao [2019] achieved an accuracy of 72.6% (3s.f.) and Chao [2018] also achieved an accuracy of 72.6% (3s.f.) on this same project and the same dataset. The accuracy of 80.7% (3s.f.) presents a considerable improvement to these.

## 7.2.2 Evaluation of Objectives

### 1. Data Preprocessing

- (a) The project has successfully devised a method of extracting two dimensional images from the three dimensional scans. This objective has been met in its entirety. The input of the model is two dimensional, and the images capture the data required to perform the classification. To further reduce the curse of dimensionality, the size of the input images was reduced to a size of  $64 \times 64$  without a detriment in performance.

### 2. CNN

- (a) The creation of the CNN was successful. the accuracy of 80.7% (3s.f.) surpasses that of AngQuo in 2020, who achieved an accuracy of 76.3%. However, the model demonstrated overfitting and failed to meet the criteria of the training and testing accuracy being within 5% of each other. This objective was potentially unachievable within the context of the project. Within most ML projects this should be aimed for and should be achievable, however the problem of having a small dataset makes it difficult to create a model that can recognise and learn the patterns to a good enough degree without overfitting because there is a lack of variety and breadth of data to train it on. This aim

could be achieved if a method of data augmentation was devised that could increase the size of the training dataset considerably.

### 3. MCI Performance

- (a) The project failed to meet the aim of having the highest class accuracy within the MCI classification. This aim is difficult to achieve, as the MCI class sits between the NC and AD classes, whereas the accuracies for NC and AD are relatively similar, at 84.4% (3 s.f.) and 85.1% (3s.f.). A more realistic aim would be for the MCI classification to meet these two accuracies and for the accuracies of all three classes to be within a small margin of each other. Attempts to meet this objective were made by investigating aspects such as class imbalance, but more MCI samples within the training data failed to solve the issue.

### 4. Small Dataset

- (a) The dataset consists of 1070 scans. This was further reduced to have an equal number of samples from each class at 243 from each, totalling a dataset of 729 samples. This goes beyond the original objective to perform the classification with even smaller a dataset. Reducing the dataset to have equal samples from each class improved the performance rather than using the whole set.

## 7.2.3 Reflection

As an aspect of personal reflection, time management and organisation have never proved an issue, however through the course of completing this project the methods of working evolved to establish more efficient processes. For example, time was saved by creating a file containing a number of functions that had been written for various purposes, like down-sampling the classes and running the k-fold validation. The easy access and organisation of these made for a lot of saved time and a reduction in time spent re-writing and testing code that had already been written. This allowed for all the model experiments that had been completed in the first semester to then be re-run on the ensemble model to ensure the changes made carried through to this case.

# Chapter 8

## Conclusion and Future Work

### 8.1 Conclusions

This project focused on the creation of a ML model that, given a three dimensional MRI brain scan, could classify it into AD, MCI or NC. The primary challenges faced here were working with small amounts of sample data, as is common in medical applications, and performing the MCI classification. Many previous works have been able to successfully classify NC and AD but the prognosis of an MCI stage is one that is most valuable to patients, as it means that treatment can be sought to prevent the development to AD.

The main stages were: establishing the data that would result from the preprocessing of the scans to be used as input to the model, and creating and determining the topology of the model itself.

Each three dimensional MRI scan went through a number of stages of preprocessing, such as linear registration to ensure that the ROI occurred at the same coordinates in each scan. Seven regions of interest were extracted that the literature pointed to demonstrating the atrophy of AD. All seven regions were included based on the indication from Gupta et al. [2019] that the more regions included, the easier the distinction to make. A two dimensional slice was taken from each scan at locations determined through experimentation, inspired by Islam and Zhang [2018] and Dai [2019].

An ensemble model was created from three models, each of which took a single plane as input. The model exhibited traditional convolutional and pooling cycles, with Leaky ReLU activation layers and three densely connected layers after flattening.

The model demonstrated an overall accuracy of 80.7% (3s.f.) and demonstrated resilience to the number of data used to train, showing success at creating a model that can learn the patterns despite limited sample sizes. The MCI classification still proved the hardest to make and it is expected that the improvement on this area will come from more careful selection of the data used as input, i.e. narrowing down the regions of the brain that are used to perform the classification.

## 8.2 Future Work

Further improvement is required to tackle the problem of the classification of MCI that this investigation aimed to improve. The ensemble model has demonstrated a benefit compared to using combined or singular planes. The further improvement of the model must come from changing the data that is input. Increasing the data set through augmentation, and by reducing the test set size for the final evaluation did not demonstrate the increase in performance that was expected. This means that the problem of working with small data sets has been solved- the model performs consistently and can learn the classification to the same standard on smaller training sets.

To produce further improvement the ROI that is used as input for the model should be further investigated. The model currently uses 7 regions combined in one image. Each of these should be investigated individually to be able to determine whether each of these is adding to or detracting from the classification. Because the stages of MCI are seen in the hippocampus first, selecting just the hippocampus as input for the model could help in the problem that this model has failed to solve- producing a comparable accuracy for the MCI classification as can be achieved in AD and NC classes.

Another avenue for further investigation could be weighting the ensembling of the three plane models based upon the models accuracy in a particular class- More complex rules could be investigated, such as considering the class accuracies of a model.

An investigation that was not performed was moving away from the default parameters of the LeakyReLU activation layer. Alpha was used at its default value of 0.3, but changing this could potentially add performance advantages. Additionally, further investigations about the relationships between dropout/batch normalisation/pooling layers should be performed to truly understand the relationships between these. This would potentially require more complex hardware or the use of cloud services.

Finally, an investigation into what the minimum size of the images could be should be conducted, as further reducing the image size will reduce the dimensionality, which could have advantages in reducing the overfitting.

# Bibliography

- Ahmed Abdulkadir, Jessica Peter, T. Brox, Olaf Ronneberger, and S. Klöppel. Voxel-based multi-class classification of ad, mci, and elderly controls: blind evaluation on an independent test set. *Proc MICCAI Workshop Challenge on Computer-Aided Diagnosis of Dementia Based on Structural MRI Data*, pages 8–15, 01 2014.
- Ethem Alpaydin. *Introduction to machine learning*. Adaptive computation and machine learning. MIT Press, Cambridge, Mass, 2nd ed edition, 2010. ISBN 9780262012430. OCLC: ocn317698631.
- P Anderer, B Saletu, B Klöppel, HV Semlitsch, and H Werner. Discrimination between demented patients and normals based on topographic eeg slow wave activity: comparison between z statistics, discriminant analysis and artificial neural network classifiers. *Electroencephalography and clinical Neurophysiology*, 91(2):108–117, 1994.
- Yuichiro Anzai. *Pattern Recognition and Machine Learning*. Elsevier Inc, 1992. ISBN 0120588307.
- Sylvain Arlot and Alain Celisse. A survey of cross-validation procedures for model selection. *Statistics Surveys*, 4(none):40 – 79, 2010. doi: 10.1214/09-SS054. URL <https://doi.org/10.1214/09-SS054>. Accessed: 2021-03-08.
- Changyong Bao. *Hyperparameter Optimisation Using Bayesian Optimisation For Medical Image Classification In Alzheimer’s Disease, Mild Cognitive Impairment And Healthy Brain*. Degree of Doctor of Philosophy, University of Manchester, Manchester, 2019.
- Richard Bellman. *Dynamic programming*. Dover Publications, Mineola, N.Y, dover ed edition, 2003. ISBN 9780486428093.
- Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013. doi: 10.1109/TPAMI.2013.50.
- James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of machine learning research*, 13(2), 2012.

- Jason Brownlee. How Do Convolutional Layers Work in Deep Learning Neural Networks?, April 2019. URL <https://machinelearningmastery.com/convolutional-layers-for-deep-learning-neural-networks/>. Accessed: 2021-04-19.
- Fang Chao. *Classification Of Medical Images Into Alzheimer's Disease, Mild Cognitive Impairment And Healthy Brain*. Degree Of Master Of Research In Infomatics, University of Manchester, Manchester, 2018.
- Bing Cheng and D. M. Titterington. Neural networks: A review from a statistical perspective. *Statistical Science*, 9(1):2–30, 1994. ISSN 08834237. URL <http://www.jstor.org/stable/2246275>. Accessed: 2021-03-09.
- Jun-Sik Choi, Eunho Lee, and Heung-Il Suk. Regional abnormality representation learning in structural mri for ad/mci diagnosis. In Yinghuan Shi, Heung-Il Suk, and Mingxia Liu, editors, *Machine Learning in Medical Imaging*, pages 64–72, Cham, 2018. Springer International Publishing. ISBN 978-3-030-00919-9.
- Claudia Cooper, Ryan Li, Constantine Lyketsos, and Gill Livingston. Treatment for mild cognitive impairment: systematic review. *The British Journal of Psychiatry*, 203(4):255–264, 2013.
- Gregory Crane. Response to “Artificial Intelligence—The Revolution Hasn’t Happened Yet”. *Harvard Data Science Review*, June 2019. doi: 10.1162/99608f92.e32f6dec. URL <https://hdsr.mitpress.mit.edu/pub/kyzf7fjv>. Accessed: 2021-03-08.
- Simon S Cross and James Cressee Elphinstone Underwood. *Underwood's pathology: a clinical approach*. Elsevier, 2013. ISBN 9780702046728 9780702046735. OCLC: 995545690.
- Shuang Dai. *A New Method For Early Diagnosis Of Alzheimer's Disease By Convolutional Neural Network, Based On 2.5D Slices Extraction*. Doctor of Philosophy, The University of Manchester, Manchester, 2019.
- Melanie Dani, Melanie Wood, Ruth Mizoguchi, Zhen Fan, Trudi Edginton, Rainer Hinz, Zarni Win, David James Brooks, and Paul Edison. Tau Aggregation Correlates with Amyloid Deposition in Both Mild Cognitive Impairment and Alzheimer's Disease Subjects. *Journal of Alzheimer's Disease*, 70(2):455–465, July 2019. ISSN 13872877, 18758908. doi: 10.3233/JAD-181168. URL <https://www.medra.org/servlet/aliasResolver?alias=iospress&doi=10.3233/JAD-181168>. Accessed: 2021-03-04.
- Arun Kumar Dubey and Vanita Jain. Comparative study of convolution neural network's relu and leaky-relu activation functions. In *Applications of Computing, Automation and Wireless Systems in Electrical Engineering*, pages 873–880. Springer, 2019.

- Prakhar Ganesh. Types of Convolution Kernels : Simplified, October 2019. URL <https://towardsdatascience.com/types-of-convolution-kernels-simplified-f040cb307c37>. Accessed: 2021-03-09.
- Ian Goodfellow. *Deep learning*. Adaptive computation and machine learning series. MIT Press, Cambridge, MA, 2016. ISBN 9780262035613.
- Anqi Guo. *A Classification Method Of Brain Images Into AD, MCI And NC*. Master of Computer Science, University of Manchester, Manchester, 2020.
- Yubraj Gupta, Ramesh Kumar Lama, Goo-Rak Kwon, and Alzheimer’s Disease Neuroimaging Initiative. Prediction and Classification of Alzheimer’s Disease Based on Combined Features From Apolipoprotein-E Genotype, Cerebrospinal Fluid, MR, and FDG-PET Imaging Biomarkers. *Frontiers in Computational Neuroscience*, 13:72, October 2019. ISSN 1662-5188. doi: 10.3389/fncom.2019.00072. URL <https://www.frontiersin.org/article/10.3389/fncom.2019.00072/full>. Accessed: 2021-03-14.
- B. D Hames and N. M Hooper. *Biochemistry*. Garland Science/Taylor & Francis Group, New York, 2011. ISBN 9780203808320 9781136667879 9780429212871. OCLC: 730500611.
- Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.
- J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, April 1982. doi: 10.1073/pnas.79.8.2554. URL <https://doi.org/10.1073/pnas.79.8.2554>. Accessed: 2021-03-09.
- Andrew P. Hunter, Lindsey R. Sheppard, Robert Karlen, and Leonardo Balietto. Artificial intelligence and national security: The importance of the ai ecosystem. Technical report, Center for Strategic and International Studies (CSIS), 2018. URL <http://www.jstor.org/stable/resrep22492.5>. Accessed: 2021-03-08.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.
- Jyoti Islam and Yanqing Zhang. Early Diagnosis of Alzheimer’s Disease: A Neuroimaging Study with Deep Learning Architectures. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1962–19622, Salt Lake City, UT, USA, June 2018. IEEE. ISBN 9781538661000. doi: 10.1109/CVPRW.2018.00247. URL <https://ieeexplore.ieee.org/document/8575412/>. Accessed: 2021-03-12.

- Amy J Jak, Stephanie Urban, ASHLEY McCAULEY, Katherine J Bangen, Lisa Delano-Wood, Jody Corey-Bloom, and Mark W Bondi. Profile of hippocampal volumes and stroke risk varies by neuropsychological definition of mild cognitive impairment. *Journal of the International Neuropsychological Society: JINS*, 15(6):890, 2009.
- Ibrahim Kandel and Mauro Castelli. The effect of batch size on the generalizability of the convolutional neural networks on a histopathology dataset. *ICT Express*, 6(4):312–315, 2020. ISSN 2405-9595. doi: <https://doi.org/10.1016/j.ict.2020.04.010>. URL <https://www.sciencedirect.com/science/article/pii/S2405959519303455>. Accessed: 2021-03-10.
- Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *CoRR*, abs/1609.04836, 2016. URL <http://arxiv.org/abs/1609.04836>. Accessed: 2021-03-10.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Parveen J. Kumar and Michael L. Clark, editors. *Clinical medicine*. Elsevier Saunders, Edinburgh, 6. ed edition, 2005. ISBN 9780702027635 9780702027642. OCLC: 835764244.
- Xiangyu Ma, Zhaoxia Li, Bin Jing, Han Liu, Dan Li, Haiyun Li, and the Alzheimer’s Disease Neuroimaging Initiative. Identify the Atrophy of Alzheimer’s Disease, Mild Cognitive Impairment and Normal Aging Using Morphometric MRI Analysis. *Frontiers in Aging Neuroscience*, 08, October 2016. ISSN 1663-4365. doi: 10.3389/fnagi.2016.00243. URL <http://journal.frontiersin.org/article/10.3389/fnagi.2016.00243/full>. Accessed: 2021-03-15.
- Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133, December 1943. ISSN 1522-9602. doi: 10.1007/BF02478259. URL <https://doi.org/10.1007/BF02478259>. Accessed: 2021-03-08.
- Katharina Morik. Medicine: Applications of machine learning. In Claude Sammut and Geoffrey I. Webb, editors, *Encyclopedia of Machine Learning*, pages 654–661. Springer US, Boston, MA, 2010. ISBN 978-0-387-30164-8. doi: 10.1007/978-0-387-30164-8\_530.
- Kevin P. Murphy. *Machine learning: a probabilistic perspective*. Adaptive computation and machine learning series. MIT Press, Cambridge, MA, 2012. ISBN 9780262018029.
- Jeannette Naish, Patricia Revest, and Denise Syndercombe Court. *Medical sciences: with Student-CONSULT access*. Saunders, Oxford, 2009. ISBN 9780702026799. OCLC: 321016330.

- Murat Oduncuoglu and Halil Kurt. Application of a neural network model for prediction of wear properties of ultrahigh molecular weight polyethylene composites. *International Journal of Polymer Science*, 2015:11, 05 2015. doi: 10.1155/2015/315710.
- Praveen Pareek. Elements of Neural Networks, September 2020. URL <https://medium.datadriveninvestor.com/elements-of-neural-networks-6fa6edeb077c>. Accessed: 2021-04-19.
- Ashish Patel. Chapter-7 Under-fitting, over-fitting and its solution, July 2018. URL <https://medium.com/ml-research-lab/under-fitting-over-fitting-and-its-solution-dc6191e34250>. Accessed: 2021-03-12.
- Adrien Payan and Giovanni Montana. Predicting alzheimer's disease: a neuroimaging study with 3d convolutional neural networks. *arXiv preprint arXiv:1502.02506*, 2015.
- Luis Perez and Jason Wang. The effectiveness of data augmentation in image classification using deep learning, 2017.
- Ronald C Petersen, Glenn E Smith, Stephen C Waring, Robert J Ivnik, Emre Kokmen, and Eric G Tangelos. Aging, memory, and mild cognitive impairment. *International psychogeriatrics*, 9 (S1):65–69, 1997.
- Lutz Prechelt. Early stopping-but when? In *Neural Networks: Tricks of the trade*, pages 55–69. Springer, 1998.
- Walter S Pritchard, Dennis W Duke, Kerry L Coburn, Norman C Moore, Karen A Tucker, Michael W Jann, and Russell M Hostetler. Eeg-based, neural-net predictive classification of alzheimer's disease versus control subjects is augmented by non-linear eeg measures. *Electroencephalography and clinical Neurophysiology*, 91(2):118–130, 1994.
- Dale Purves, George J. Augustine, David Fitzpatrick, William C. Hall, Anthony-Samuel LaMantia, and Leonard E. White. *Neuroscience*, volume 5. Sinauer Associates, 2012. ISBN 978-0-87893-695-3.
- Saima Rathore, Mohamad Habes, Muhammad Aksam Iftikhar, Amanda Shacklett, and Christos Davatzikos. A review on neuroimaging-based classification studies and associated feature extraction methods for Alzheimer's disease and its prodromal stages. *NeuroImage*, 155:530–548, July 2017. ISSN 10538119. doi: 10.1016/j.neuroimage.2017.03.057. URL <https://linkinghub.elsevier.com/retrieve/pii/S1053811917302823>. Accessed: 2021-03-15.
- David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature (London)*, 323(6088):533–536, 1986. ISSN 0028-0836.

- P.Y Simard, D Steinkraus, and J.C Platt. Best practices for convolutional neural networks applied to visual document analysis. In *Seventh International Conference on Document Analysis and Recognition, 2003. Proceedings*, pages 958–963. IEEE, 2003. ISBN 9780769519609.
- Glenn E. Smith. *Mild cognitive impairment and dementia : definitions, diagnosis, and treatment*. Oxford workshop series / American Academy of Clinical Neuropsychology. Oxford University Press, New York, 2013. ISBN 9780199909155.
- Francisco J Solis and Roger J.-B Wets. Minimization by random search techniques. *Mathematics of operations research*, 6(1):19–30, 1981. ISSN 0364-765X.
- S. Sudha, K. B. Jayanthi, C. Rajasekaran, and T. Sunder. Segmentation of roi in medical images using cnn- a comparative study. In *TENCON 2019 - 2019 IEEE Region 10 Conference (TENCON)*, pages 767–771, 2019. doi: 10.1109/TENCON.2019.8929648.
- V. Sze, Y. Chen, T. Yang, and J. S. Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017. doi: 10.1109/JPROC.2017.2761740.
- Shui-Hua Wang, Preetha Phillips, Yuxiu Sui, Bin Liu, Ming Yang, and Hong Cheng. Classification of alzheimer’s disease based on eight-layer convolutional neural network with leaky rectified linear unit and max pooling. *Journal of medical systems*, 42(5):1–11, 2018.
- Lei Wu, Zhanxing Zhu, and Weinan E. Towards understanding generalization of deep learning: Perspective of loss landscapes. *CoRR*, abs/1706.10239, 2017. URL <http://arxiv.org/abs/1706.10239>. Accessed: 2021-03-09.
- Barbara Young, Geraldine O’Dowd, and Phillip Woodford. *Wheater’s functional histology: a text and colour atlas*. Churchill Livingstone/Elsevier, Philadelphia, PA, sixth edition edition, 2014. ISBN 9780702047473.
- Xiaohu Zhang, Yuexian Zou, and Wei Shi. Dilated convolution neural network with leakyrelu for environmental sound classification. In *2017 22nd International Conference on Digital Signal Processing (DSP)*, pages 1–5. IEEE, 2017.

# Appendix A

## 2.5D Data Extraction

Extraction of multiple slices from each plane:

```
#
#####

# This program takes the MRI scan images as input, they are nii.gz type.
# It reads each and transforms into a 2D RGB image.
# I have used python features to parallelise the image processing.

# Anna Lackey

#
#####

# Imports
-----
import numpy as np
import os
import nibabel as nib
import imageio
from PIL import Image
import multiprocessing
from multiprocessing import Pool
import time

# File Paths
-----
LABEL_FILE = "/home/anna/Documents/thirdyearproject/data/MRI_data/label_all.
txt"
MRI_FOLDER = "/home/anna/Documents/thirdyearproject/data/MRI_data/PreOut"
OUTPUT = "/home/anna/Documents/thirdyearproject/data/PNG_data/multiple_planes"
```

```

# Global Variables
-----

# How much of the image is cropped
CROP_FACTOR = 91

# Final size of the image
IMG_SIZE_X = 64
IMG_SIZE_Y = 64

# locations (in voxels) where slicing occurs along each axis
XSTART = 56
XEND = 68
YSTART = 110
YEND = 122
ZSTART = 47
ZEND = 59

# step made- This, along with the Image bounds for each axis determines the
# number of centre points taken for the final image
# original value was 2, which resulted in 18 selected centre points
STEP = 2
print(f'Step = {STEP}')

# Time it
start_time = time.time()

# Perform the Slice Extraction
-----

# Given a 3D .nii.gz MRI Scanned image, this function extracts slices accross
# each of the three dimensional axis given a start and end point along the
# axis.
# It accumulates the slices to produce an image for each plane which are
# encoded into a single image using the red green and blue channels.
def nii_to_image(file):

    # start reading .nii file
    img_path = os.path.join(MRIFOLDER, file)
    img = nib.load(img_path)
    img_fdata = img.get_fdata()
    fname = file.replace('.nii.gz', '') # Remove .nii suffix
    a = fname.split('_')
    split_fname = '_'.join(a[0:5]) #get key label name

    imgfname = labelcompare(split_fname)

    #Coronal plane(xz) Sagittal plane(yz) Transverse plane(xy)

```

```

# setting the stack as the first slices
xy_stack = img_fdata[:, :, ZSTART]
xz_stack = img_fdata[:, YSTART, :]
yz_stack = img_fdata[XSTART, :, :]

# Accumulate slices for each physical plane:

# z 47-59 ,5 vary z
xy_loops = 1
for i in range(ZSTART + STEP, ZEND, STEP): # range z
    img_xy = img_fdata[:, :, i]
    xy_stack = np.add(xy_stack, img_xy)
    xy_loops += 1

# y 110-122 vary y
xz_loops = 1
for i in range(YSTART + STEP, YEND, STEP): # range y
    img_xz = img_fdata[:, i, :]
    xz_stack = np.add(xz_stack, img_xz)
    xz_loops += 1

# x 56-67 vary x (THIS ISNT 18 centre points!? This is 5 steps not 6?)
yz_loops = 1
for i in range(XSTART + STEP, XEND, STEP): # range x
    img_yz = img_fdata[i, :, :]
    yz_stack = np.add(yz_stack, img_yz)
    yz_loops += 1

# average the stacks
xy_stack = xy_stack / xy_loops
xz_stack = xz_stack / xz_loops
yz_stack = yz_stack / yz_loops

# transform to images
slice_xy = MatrixToImage(xy_stack)
slice_xz = MatrixToImage(xz_stack)
slice_yz = MatrixToImage(yz_stack)

# crop the border, or whitespace, around each image
slice_xy_cut = crop_image(slice_xy)
slice_xz_cut = crop_image(slice_xz)
slice_yz_cut = crop_image(slice_yz)

# combine the three images in the R, G, B channels

```

```

rgb_array = np.zeros((img_yz.shape[1], img_yz.shape[1], 3), "uint8")
rgb_array[:, :, 0], rgb_array[:, :, 1], rgb_array[:, :, 2] = slice_yz_cut,
slice_xz_cut, slice_xy_cut
rgb_image = Image.fromarray(rgb_array)

# Resize to reduce processing power
rgb_image_tra = resize_image(rgb_image)

# Write to output folder
imageio.imwrite(f"{OUTPUT}/{imgfname}.png", rgb_image_tra)

# Add Labels to File names
-----
def labelcompare(imgname):
    readlabel = open(LABEL_FILE)
    i = 0

    for eachLine in readlabel:
        i = i + 1
        a = eachLine.split()
        label = a[0:1]

        if label[0] == imgname:
            a = eachLine.split()
            b = a[1:2]
            img_f_name = '_' .join(b + label)
            return img_f_name

    return "-1"

# Matrix to Image
-----
def MatrixToImage(data):
    data = data*255
    new_im = Image.fromarray(data.astype(np.uint8))
    return new_im

# Crop Image- removes the border by CROP_FACTOR
-----
def crop_image(image):
    half_the_width = image.size[0] / 2
    half_the_height = image.size[1] / 2
    image_new = image.crop(
        (
            half_the_width - CROP_FACTOR,
            half_the_height - CROP_FACTOR,
            half_the_width + CROP_FACTOR,

```

```

        half_the_height + CROP_FACTOR
    )
)
return image_new

# resize image to 64 x 64
-----
# I originally cut this out. but after running some experiments I found it
# sped things up a lot later on and didnt affect the accuracy too much
def resize_image(image):
    out = image.resize((IMG_SIZE_X,IMG_SIZE_Y),Image.ANTIALIAS)
    return out

# Main - parallelise the process
-----
if __name__ == '__main__':

    # make directory if doesnt exist
    if not os.path.exists(OUTPUT):
        os.mkdir(OUTPUT)

    no_cores = multiprocessing.cpu_count()
    print("Running on %s cores" % no_cores)
    processPool = Pool(no_cores)
    processPool.map(nii_to_image , os.listdir(MRI_FOLDER))
    print("Run Time: --- %s seconds ---" % (time.time() - start_time))

Extraction single slice from each plane:

#
#####

# This program takes the MRI scan images as input, they are nii.gz type.
# It reads each and transforms into a 2D RGB image.
# I have taken ShuangDais code and improved it by reducing its complexity.
# I have used python features to parallelise the image processing.

# Anna Lackey

#
#####

# Imports
-----
import numpy as np
import os

```

```

import sys
import nibabel as nib
import imageio
from PIL import Image
import multiprocessing
from multiprocessing import Pool
import time

# File Paths
-----
LABEL_FILE = "/home/anna/Documents/thirdyearproject/data/MRI_data/label_all.
txt"
MRI_FOLDER = "/home/anna/Documents/thirdyearproject/data/MRI_data/PreOut"
OUTPUT_PNG = "/home/anna/Documents/thirdyearproject/data/Final/PNG"

# Global Vars
-----
# How much of the image is cropped
CROP_FACTOR = 91

# locations (in voxels) where slicing occurs
XSLICE = 72
YSLICE = 118
ZSLICE = 63

# resize
IMG_SIZE_X = 64
IMG_SIZE_Y = 64

# step made- the number of centre points.
# original value was 2, which resulted in 18 selected centre points
STEP = 2
print(f'Step = {STEP}')

# Time it
start_time = time.time()

# Bulk programming - changed nothing here
-----
def nii_to_image(file):
    # read .nii file
    img_path = os.path.join(MRI_FOLDER, file)
    img = nib.load(img_path)
    img_fdata = img.get_fdata()

```

```

# get image file name - add the label
fname = file.replace('.nii.gz', '') # Remove .nii suffix
a = fname.split('_')
split_fname = '_'.join(a[0:5]) #get key label name

imgfname = labelcompare(split_fname)

#Coronal plane(xz) Sagittal plane(yz) Transverse plane(xy)

# sagittal plane
img_yz = img_fdata[XSLICE, :, :]
slice_yz = MatrixToImage(img_yz)
slice_yz_cut = crop_image(slice_yz)

# coronal plane
img_xz = img_fdata[:, YSLICE, :]
slice_xz = MatrixToImage(img_xz)
slice_xz_cut = crop_image(slice_xz)

# transverse plane
img_xy = img_fdata[:, :, ZSLICE]
slice_xy = MatrixToImage(img_xy)
slice_xy_cut = crop_image(slice_xy)

# merge the images to RGB
rgb_array = np.zeros((img_yz.shape[1], img_yz.shape[1], 3), "uint8")
rgb_array[:, :, 0], rgb_array[:, :, 1], rgb_array[:, :, 2] = slice_yz_cut,
slice_xz_cut, slice_xy_cut
rgb_image = Image.fromarray(rgb_array)

rgb_image = resize_image(rgb_image)

# save image
imageio.imwrite(f"{OUTPUT_PNG}/{imgfname}.png", rgb_image)

# Add Labels to File names
-----

def labelcompare(imgname):
    readlabel = open(LABEL_FILE)
    i = 0

    for eachLine in readlabel:
        i = i + 1
        a = eachLine.split()
        label = a[0:1]

```

```

        if label[0] == imgname:
            a = eachLine.split()
            b = a[1:2]
            img_f_name = '_'.join(b + label)
            return img_f_name

# Matrix to Image
-----

def MatrixToImage(data):
    data = data*255
    new_im = Image.fromarray(data.astype(np.uint8))
    return new_im

# Crop Image- remove blackspace
-----

def crop_image(image):
    half_the_width = image.size[0] / 2
    half_the_height = image.size[1] / 2
    image_new = image.crop(
        (
            half_the_width - CROP_FACTOR,
            half_the_height - CROP_FACTOR,
            half_the_width + CROP_FACTOR,
            half_the_height + CROP_FACTOR
        )
    )
    return image_new

# resize image to 64 x 64
-----

# I originally cut this out. but my GPU ran out of memory working with 182
#x182
def resize_image(image):
    out = image.resize((IMG_SIZE_X, IMG_SIZE_Y), Image.ANTIALIAS)
    return out

# Main - parallelise the process
-----

if __name__ == '__main__':
    # make directory if doesnt exist
    if not os.path.exists(OUTPUT_PNG):
        os.mkdir(OUTPUT_PNG)

```

```
# Generate PNGs in Parallel
no_cores = multiprocessing.cpu_count()
print("Running on %s cores" % no_cores)
processPool = Pool(no_cores)
processPool.map(nii_to_image, os.listdir(MRI_FOLDER))
print("Run Time: --- %s seconds ---" % (time.time() - start_time))
```

# Appendix B

## Adam Algorithm

---

**Algorithm 1:** *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation.  $g_t^2$  indicates the elementwise square  $g_t \odot g_t$ . Good default settings for the tested machine learning problems are  $\alpha = 0.001$ ,  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$  and  $\epsilon = 10^{-8}$ . All operations on vectors are element-wise. With  $\beta_1^t$  and  $\beta_2^t$  we denote  $\beta_1$  and  $\beta_2$  to the power  $t$ .

---

**Require:**  $\alpha$ : Stepsize

**Require:**  $\beta_1, \beta_2 \in [0, 1)$ : Exponential decay rates for the moment estimates

**Require:**  $f(\theta)$ : Stochastic objective function with parameters  $\theta$

**Require:**  $\theta_0$ : Initial parameter vector

$m_0 \leftarrow 0$  (Initialize 1<sup>st</sup> moment vector)

$v_0 \leftarrow 0$  (Initialize 2<sup>nd</sup> moment vector)

$t \leftarrow 0$  (Initialize timestep)

**while**  $\theta_t$  not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$  (Get gradients w.r.t. stochastic objective at timestep  $t$ )

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$  (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$  (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  (Update parameters)

**end while**

**return**  $\theta_t$  (Resulting parameters)

---

[Kingma and Ba, 2014]

# Appendix C

## Initial Model

```
def model():
    my_model = Sequential([
        Conv2D(16, kernel_size=3, activation='relu',
              input_shape=(64,64,1)),
        Conv2D(16, kernel_size=3, activation='relu'),
        MaxPooling2D(pool_size=2),
        Conv2D(32, kernel_size=3, activation='relu'),
        Conv2D(64, kernel_size=3, activation='relu'),
        MaxPooling2D(pool_size=2),
        Dropout(0.15),

        Flatten(),
        Dense(448, activation='tanh'),
        Dropout(0.25),
        Dense(3, activation='softmax')
    ])

    my_model.compile(
        optimizer=optimizers.Adam(learning_rate=0.00013607),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )

    return my_model
```

# Appendix D

## Initial Tuner Model

```
from kerastuner import HyperModel
from tensorflow import keras
from tensorflow.keras.layers import Conv2D, Dense, Dropout
from tensorflow.keras.layers import Flatten, MaxPooling2D
import os
import shutil
```

```
PROJECT_NAME = "Output"
```

```
class CNNModel(HyperModel):
    def __init__(self, input_shape, num_classes):
        self.input_shape = input_shape
        self.num_classes = num_classes

    def build(self, hp):
        model = keras.Sequential()

        # input layer
        model.add(
            Conv2D(
                filters=hp.Int(
                    "filers_layer_one", min_value=16
                    , max_value=128
                    , step=32, default=16
                ),
                kernel_size=hp.Choice(
                    "kernel_size_one", values=[3,5,7]
                    , default=3
                ),
                activation=hp.Choice(
                    "layer_one_activation",
                    values=["relu", "tanh", "sigmoid"],

```

```

        default="relu",
    ),
    input_shape=self.input_shape
)
)

# conv pool layers
for i in range(0, hp.Int(
    "i", min_value=1, max_value=3, step=1
    , default=2
    )):

    model.add(
        Conv2D(
            filters=hp.Int(
                f"cycle{i}_filters", min_value=16
                , max_value=128
                , step=32, default=32
            ),
            kernel_size=hp.Choice(
                f"cycle{i}_kernel", values=[3,5,7]
                , default=3
            ),
            activation=hp.Choice(
                f"cycle{i}_activation",
                values=["relu", "tanh", "sigmoid"],
                default="relu",
            )
        )
    )

# pool
model.add(MaxPooling2D(pool_size=hp.Choice(
    f"cycle{i}_pool", values=[1,2], default=2
    )
))

# drop
model.add(
    Dropout(
        rate=hp.Float(
            f"cycle{i}_dropout", min_value=0.0
            , max_value=0.4
            , default=0.25, step=0.05,
        )
    )
)

```

```

# flat
model.add(Flatten())

# dense and drop layers
for k in range(0, hp.Int(
    "k", min_value=1, max_value=3, step=1, default=2
)):
    model.add(
        Dense(
            units=hp.Int(
                f"denseDrop{k}_units", min_value=32
                , max_value=512
                , step=32, default=128
            ),
            activation=hp.Choice(
                f"denseDrop{k}_denseActivation",
                values=["relu", "tanh", "sigmoid"],
                default="relu",
            ),
        )
    )
    model.add(
        Dropout(
            rate=hp.Float(
                f"denseDrop{k}_dropout", min_value=0.0
                , max_value=0.4
                , default=0.25, step=0.05
            )
        )
    )

# output layer
model.add(Dense(self.num_classes, activation="softmax"))

print(model.summary())

# compile
model.compile(
    optimizer=keras.optimizers.Adam(
        hp.Float(
            "learning_rate",
            min_value=1e-4,
            max_value=1e-2,
            sampling="LOG",
            default=1e-3,
        )
    ),
    loss="categorical_crossentropy",

```

```
        metrics=["accuracy"],  
    )  
    return model
```

# Appendix E

## Ensemble Methods

```
def averaging_ensemble(red_model, green_model
                        , blue_model, testX, testy):

    testX_red = testX[:, :, :, 0]
    testX_red = np.expand_dims(testX_red, axis=-1)

    testX_green = testX[:, :, :, 1]
    testX_green = np.expand_dims(testX_green, axis=-1)

    testX_blue = testX[:, :, :, 2]
    testX_blue = np.expand_dims(testX_blue, axis=-1)

    red_predict = red_model.predict(testX_red)
    green_predict = green_model.predict(testX_green)
    blue_predict = blue_model.predict(testX_blue)

    yhats = [red_predict, green_predict, blue_predict]

    yhats = np.array(yhats)

    # sum across ensemble members
    summed = np.sum(yhats, axis=0)

    # argmax across classes
    result = np.argmax(summed, axis=1)
    result = accuracy_score(testy, result)
    return result

def weighted_ensemble(red_model, green_model, blue_model
                      , testX, testy, weights):

    testX_red = testX[:, :, :, 0]
```

```

testX_red = np.expand_dims(testX_red , axis=-1)

testX_green = testX[:, :, :, 1]
testX_green = np.expand_dims(testX_green , axis=-1)

testX_blue = testX[:, :, :, 2]
testX_blue = np.expand_dims(testX_blue , axis=-1)

red_predict = red_model.predict(testX_red)
green_predict = green_model.predict(testX_green)
blue_predict = blue_model.predict(testX_blue)

yhats = [red_predict , green_predict , blue_predict]

yhats = np.array(yhats)

# sum across ensemble members by weights
summed = tensordot(yhats , weights , axes=((0) ,(0)))

# argmax across classes
result = np.argmax(summed , axis=1)
result = accuracy_score(testy , result)
return result

```

# Appendix F

## Models Resulting From Random Search

```
def red():
    my_model = Sequential([
        Conv2D(48, kernel_size=7, activation='tanh', input_shape=(64,64,1)),
        Conv2D(16, kernel_size=7, activation='relu'),
        MaxPooling2D(pool_size=2),
        Dropout(0.3),
        Conv2D(80, kernel_size=5, activation='tanh'),
        Dropout(0.4),
        Conv2D(48, kernel_size=7, activation='tanh'),
        MaxPooling2D(pool_size=2),
        Dropout(0.35),

        Flatten(),
        Dense(448, activation='tanh'),
        Dense(320, activation='sigmoid'),
        Dropout(0.05),
        Dense(96, activation='relu'),
        Dropout(0.2),
        Dense(3, activation='softmax')
    ])

    my_model.compile(
        optimizer=optimizers.Adam(learning_rate=0.0005648786702753176),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    # batch 128
    # epochs 30

    return my_model

def green():
```

```

my_model = Sequential([
    Conv2D(112, kernel_size=3, activation='relu', input_shape=(64,64,1)),
    Conv2D(112, kernel_size=3, activation='relu'),
    MaxPooling2D(pool_size=2),
    Dropout(0.1),
    Conv2D(80, kernel_size=3, activation='tanh'),
    MaxPooling2D(pool_size=2),
    Dropout(0.05),
    Conv2D(48, kernel_size=5, activation='relu'),
    Dropout(0.05),

    Flatten(),
    Dense(64, activation='tanh'),
    Dropout(0.35),
    Dense(224, activation='relu'),
    Dropout(0.2),
    Dense(3, activation='softmax')
])

my_model.compile(
    optimizer=optimizers.Adam(learning_rate=0.0012341315610031297),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
# batch 32
# epochs 45

return my_model

def blue():
    my_model = Sequential([
        Conv2D(48, kernel_size=5, activation='tanh', input_shape=(64,64,1)),
        Conv2D(48, kernel_size=3, activation='tanh'),
        Dropout(0.2),
        Conv2D(80, kernel_size=3, activation='relu'),
        MaxPooling2D(pool_size=2),
        Dropout(0.35),
        Conv2D(48, kernel_size=3, activation='tanh'),
        MaxPooling2D(pool_size=2),
        Dropout(0.25),

        Flatten(),
        Dense(224, activation='relu'),
        Dropout(0.25),
        Dense(448, activation='tanh'),
        Dropout(0.05),
        Dense(256, activation='sigmoid'),
        Dropout(0.2),

```

```
        Dense(3, activation='softmax')
    ])

my_model.compile(
    optimizer=optimizers.Adam(learning_rate=0.0011684126260235346),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
# batch 128
# epochs 50

return my_model
```

# Appendix G

## Model Resulting from Conv Depth Experiment

```
def model():
    my_model = Sequential([
        Conv2D(32, kernel_size=3, activation='relu', input_shape=(64,64,1),
data_format='channels_last'),
        Conv2D(64, kernel_size=3, activation='relu'),
        MaxPooling2D(pool_size=2),
        Conv2D(256, kernel_size=3, activation='relu'),
        MaxPooling2D(pool_size=2),

        Conv2D(256, kernel_size=3, activation='relu'),
        BatchNormalization(),
        MaxPooling2D(pool_size=2),

        Flatten(),
        Dense(448, activation='tanh'),
        Dropout(0.25),
        Dense(3, activation='softmax')
    ])

    my_model.compile(
        optimizer=optimizers.Adam(learning_rate=0.00013607),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )

    # batch_size=32
    # epochs = 40
    return my_model
```

## Appendix H

# Model Resulting from Dense Layer Depth Experiment

```
def model():
    my_model = Sequential([
        Conv2D(32, kernel_size=3, activation='relu', input_shape=(64,64,1),
data_format='channels_last'),
        Conv2D(64, kernel_size=3, activation='relu'),
        MaxPooling2D(pool_size=2),
        Conv2D(256, kernel_size=3, activation='relu'),
        MaxPooling2D(pool_size=2),

        Conv2D(256, kernel_size=3, activation='relu'),
        BatchNormalization(),
        MaxPooling2D(pool_size=2),

        Flatten(),
        Dense(410, activation='tanh'),
        Dense(410, activation='tanh'),
        Dense(410, activation='tanh'),
        Dropout(0.25),
        Dense(3, activation='softmax')
    ])

    my_model.compile(
        optimizer=optimizers.Adam(learning_rate=0.00013607),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )

    # batch_size=32
    # epochs = 40
```

```
return my_model
```

# Appendix I

## Final Model

```
model = Sequential([
    Conv2D(32, kernel_size=3, input_shape=(64,64,1),
        data_format='channels_last',
        padding='same'),
    LeakyReLU(),
    Conv2D(64, kernel_size=3, padding='same'),
    LeakyReLU(),
    MaxPooling2D(pool_size=3),
    Conv2D(256, kernel_size=3, padding='same'),
    LeakyReLU(),
    MaxPooling2D(pool_size=3),
    Conv2D(256, kernel_size=3, padding='same'),
    LeakyReLU(),
    BatchNormalization(),
    MaxPooling2D(pool_size=2),

    Flatten(),
    Dense(410),
    LeakyReLU(),
    Dense(410),
    LeakyReLU(),
    Dense(410),
    LeakyReLU(),
    Dropout(0.25),
    Dense(3, activation='softmax')
])

model.compile(
    optimizer=optimizers.Adam(learning_rate=0.00005),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

## Model Summary:

Model: "sequential"

| Layer (type)                              | Output Shape        | Param # |
|-------------------------------------------|---------------------|---------|
| conv2d (Conv2D)                           | (None, 64, 64, 32)  | 320     |
| leaky_re_lu (LeakyReLU)                   | (None, 64, 64, 32)  | 0       |
| conv2d_1 (Conv2D)                         | (None, 64, 64, 64)  | 18496   |
| leaky_re_lu_1 (LeakyReLU)                 | (None, 64, 64, 64)  | 0       |
| max_pooling2d (MaxPooling2D)              | (None, 21, 21, 64)  | 0       |
| conv2d_2 (Conv2D)                         | (None, 21, 21, 256) | 147712  |
| leaky_re_lu_2 (LeakyReLU)                 | (None, 21, 21, 256) | 0       |
| max_pooling2d_1 (MaxPooling2D)            | (None, 7, 7, 256)   | 0       |
| conv2d_3 (Conv2D)                         | (None, 7, 7, 256)   | 590080  |
| leaky_re_lu_3 (LeakyReLU)                 | (None, 7, 7, 256)   | 0       |
| batch_normalization (Batch Normalization) | (None, 7, 7, 256)   | 1024    |
| max_pooling2d_2 (MaxPooling2D)            | (None, 3, 3, 256)   | 0       |
| flatten (Flatten)                         | (None, 2304)        | 0       |
| dense (Dense)                             | (None, 410)         | 945050  |
| leaky_re_lu_4 (LeakyReLU)                 | (None, 410)         | 0       |
| dense_1 (Dense)                           | (None, 410)         | 168510  |
| leaky_re_lu_5 (LeakyReLU)                 | (None, 410)         | 0       |
| dense_2 (Dense)                           | (None, 410)         | 168510  |
| leaky_re_lu_6 (LeakyReLU)                 | (None, 410)         | 0       |
| dropout (Dropout)                         | (None, 410)         | 0       |
| dense_3 (Dense)                           | (None, 3)           | 1233    |
| Total params: 2,040,935                   |                     |         |

Trainable params: 2,040,423  
Non-trainable params: 512

# Acronyms

**AD** Alzheimer’s Disease. 7–10, 12–17, 19, 24, 26, 27, 29–32, 43, 45, 48, 50, 52, 62, 63, 71–73, 75–77

**ADNI** Alzheimer’s Disease Neuroimaging Initiative. 13, 14, 29, 33

**APP** amyloid precursor protein. 12

**CNN** Convolutional Neural Network. 9–11, 18, 20–22, 27, 29, 74

**DNN** Deep Neural Network. 5, 20, 27

**GM** Grey Matter. 27, 28

**GMV** Grey Matter Volume. 27

**IDE** Integrated Development Environment. 38

**MCI** Mild Cognitive Impairment. 1, 8–10, 13–17, 19, 24, 27, 29–32, 43, 45, 48–50, 52, 54, 61–63, 67, 71–73, 75–77

**ML** Machine Learning. 8–10, 16–18, 20, 21, 26–29, 53, 74, 76

**MRI** Magnetic Resonance Imaging. 5, 8, 9, 14, 17, 19, 26–31, 34, 35, 37, 40, 74, 76

**NC** Normal Cognition. 8, 9, 13, 15–17, 19, 27, 29–31, 43, 45, 48, 50, 52, 61–63, 72, 73, 75–77

**ReLU** Rectified Linear Unit. 22

**ROI** Regions of Interest. 27–29, 74, 76, 77

**s.f.** Significant Figures. 1, 7, 51, 60, 63, 74–76